

3 State of the art

Thermal imaging, or infrared thermography, is a non-contact, passive technology that lets you visualize temperature differences across objects and environments. It works by picking up the far-infrared radiation that all matter above absolute zero naturally gives off. Specialized sensors, often micro-thermopile arrays, pick up these invisible heat signatures and convert them into tiny, continuous voltage variations. Those signals are then processed and turned into a visible heat map, called a thermogram.

This technology used to be locked behind expensive military and industrial applications, but recent progress in micro-sensors has made it far more accessible. The goal of this thesis is to build on these advances to design and physically build a fully autonomous, portable, and affordable thermal camera, an easy-to-use diagnostic tool that works entirely on its own, without needing external peripherals. By keeping the design compact, giving it solid battery life, and providing real-time visual feedback, this project aims to turn raw thermal data into a practical, standalone device that can actually be used in real-world diagnostic settings.

3.1 Thermography

At its core, thermography is the science of visualizing the invisible. Our eyes only pick up a narrow sliver of the electromagnetic spectrum, what we call visible light. But most of the thermal energy given off by physical objects falls completely outside that range. Infrared radiation sits just beyond the red edge of what we can see, stretching from around $0.7\ \mu\text{m}$ up to $1000\ \mu\text{m}$ [1].

We can't see this radiation, but we feel it all the time as heat. A thermal imaging system essentially works as a real-time translator: it picks up these long-wavelength emissions and maps them onto a visible color scale we can interpret. That's what lets operators spot things like mechanical friction, electrical resistance, or thermal leaks with their own eyes and catch them long before they turn into serious hardware failures [2].

3.1.1 Physical Laws

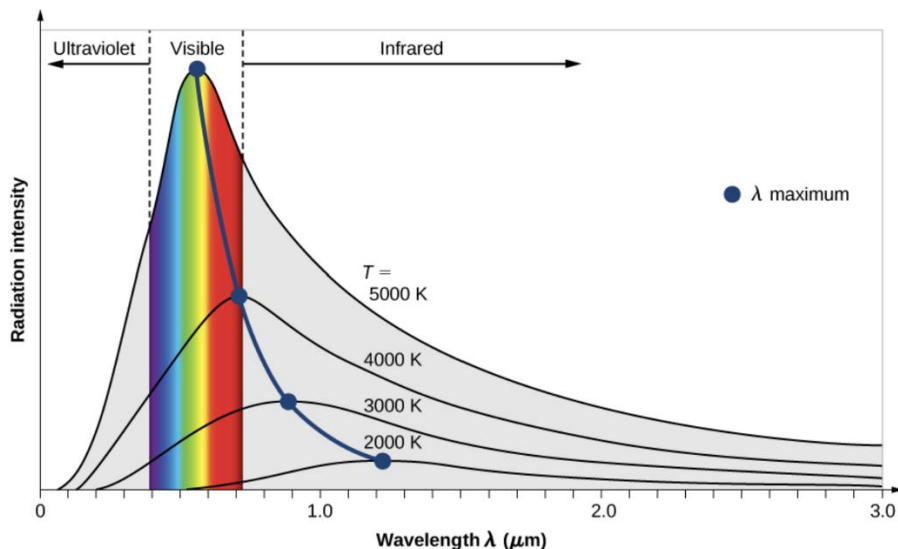


Figure 5. Blackbody Radiation Curves and Wien's Displacement Law Shift [3]

At its core, thermography works because every object with a temperature above absolute zero (0 K, or -273.15°C) constantly gives off thermal radiation as electromagnetic waves. How that radiation behaves and how heat gets generated in the first place comes down to a handful of well-established laws from thermodynamics, quantum mechanics, and electromagnetism [1].

Planck's Law: Planck's Law describes how much radiation a black body in thermal equilibrium gives off at each wavelength, based purely on its temperature. In practical terms, it shows how the energy spectrum shifts as temperature rises and sets the theoretical ceiling on infrared emission for any given thermal state [3].

You can see this in Figure 5: as an object's temperature climbs, the total energy it emits across all wavelengths shoots up, and the peak of the curve slides toward shorter wavelengths [3].

Wien's Displacement Law: This one follows directly from Planck's Law and pins down exactly where that peak emission wavelength sits for a given temperature the shift visible in Figure 5. The relationship is simple: peak wavelength is inversely proportional to the object's absolute temperature, given by relation:

$$\lambda_{max} = \frac{b}{T} \quad (1)$$

where:

λ_{max} – the wavelength corresponding to the maximum emission intensity [μm];

b – Wien's displacement constant, approximately equal to $2898 \mu\text{m} \cdot \text{K}$;

T – the absolute temperature of the target object [K].

It's this law that explains why hot objects radiate at much shorter wavelengths, sometimes edging into the visible range, while cooler objects stay confined to the long-wave infrared band [3].

Joule's First Law (Heat Generation): In a lot of industrial and electronics diagnostics, what you're measuring isn't radiation from some external source, it's electrical power turning into heat. When current flows through a resistive conductor, the heat it produces follows Joule's law, given by relation:

$$P = I^2 R = VI \quad (2)$$

where:

P – the generated thermal power [W];

I – the electrical current [A];

R – the electrical resistance of the conductor [Ω];

V – the voltage drop across the component [V].

This matters for two reasons: it helps diagnose external electrical faults, but it's just as important for managing self-heating inside the embedded single-board computer itself, so internal heat doesn't throw off local sensor readings [4].

3.1.2 Optoelectronic Conversion

Turning these theoretical emissions into actual, usable data is where microelectromechanical systems (MEMS) come in. Uncooled thermal arrays are essentially a grid of microscopic thermopile sensors. Unlike a regular camera, which captures visible-light photons, these sensors pick up far-infrared radiation through thermoelectricity instead.

When far-infrared radiation hits the active junction of a thermopile, the sensor absorbs that heat and, through the Seebeck effect, produces a tiny voltage difference relative to a reference junction. That small, continuous voltage swing is really the whole foundation the system runs on.

By reading these micro-voltages across the array, the embedded system gets the raw differential data it needs to work out the absolute temperature of whatever it's looking at [5].

3.2 Practical Applications of Thermography

3.2.1 Predictive Electrical Maintenance and Diagnostics

In electrical engineering, infrared thermography has become an essential non-contact tool for preventative maintenance and catching faults early. Electrical systems are bound by the physics of Joule heating, so any rise in resistance naturally means localized heat. When components like circuit breakers, busbars, transformers, or high-voltage cable connections start to degrade from corrosion, loose contacts, or heavy phase imbalances, their local resistance spikes. That spike shows up as a distinct thermal hot-spot, invisible to the naked eye, but impossible to miss on a thermographic image. By running routine thermal scans on switchgear and panels under active load (typically above 40% of rated capacity), technicians can catch these anomalies well before they turn into a catastrophic failure. Catching problems this early cuts down on unplanned downtime and lowers the risk of electrical fires caused by thermal runaway [6].

3.2.2 Building Diagnostics and Energy Audits

Infrared thermography has also become a cornerstone of civil engineering and building envelope analysis, used to assess both structural integrity and energy efficiency. Buildings are constantly exchanging heat with the outside environment, and that exchange can be measured quantitatively across the building's envelope. With passive thermal imaging, auditors can quickly spot thermal bridges, locate missing or degraded insulation, and find the micro-fractures responsible for costly energy drafts. It's also remarkably effective at catching moisture that's crept beneath the surface of concrete roofs or inside drywall cavities. Since water holds heat much longer than typical building materials do, spots with trapped moisture heat up and cool down more slowly than the dry areas around them. During temperature swings, that difference shows up as a distinct cooler patch on the thermogram, which lets inspectors track down leaks and mold risks without tearing anything apart [7].

3.2.3 Mechanical System Condition Monitoring

In heavy manufacturing and industrial settings, thermography is widely used to keep tabs on high-speed rotating machinery. Components like industrial motors, drive belts, gearboxes, and shaft bearings all generate some friction just from normal operation. But when something's wrong, shaft misalignment, poor lubrication, worn-out bearings, that friction climbs sharply, and so does the heat. By first establishing a baseline thermal profile for a healthy machine, technicians can then spot abnormal temperature rises while the equipment is running. Heat tends to show up before vibration or noise does, which makes thermal cameras a genuinely useful early-warning tool for condition-based maintenance (CBM). Catching these micro-frictional issues early means operators can schedule repairs during planned downtime instead of dealing with an unplanned breakdown, stretching the life of expensive equipment in the process [8].

3.3 Sensor Technology

For a long time, high-performance thermal imaging depended on quantum photon detectors, which needed bulky cryogenic cooling just to keep the sensor's own heat from throwing off its infrared readings. Modern embedded thermography has largely moved past that, today it relies almost entirely on uncooled thermal detectors built with Micro-Electro-Mechanical Systems (MEMS) technology, usually in the form of microbolometers or micro-thermopiles. Instead of counting individual infrared photons, these sensors work by measuring the actual temperature change in a microscopic absorber element when far-infrared radiation hits it. In micro-thermopile designs specifically, that absorbed heat creates a tiny temperature gradient across two different conductive materials, which generates a proportional voltage through the Seebeck effect. String together hundreds or thousands of these microscopic sensors into a grid, called a Focal Plane Array (FPA), and you get a system that can sample incident radiation across an entire field of view at once, turning that array of differential voltages into a continuous two-dimensional thermal map [9].

What's really driven the development of these uncooled FPAs is an industry-wide push to shrink Size, Weight, Power, and Cost which is exactly what's turned thermal cameras from massive, expensive military hardware into something you can buy as a small consumer module [9].

Advances in silicon wafer-level vacuum packaging and CMOS (Complementary Metal-Oxide-Semiconductor) fabrication have let engineers shrink the pixel pitch, the physical spacing between sensor elements, dramatically. Smaller pixels mean higher-resolution arrays packed into a much smaller footprint. On top of that, modern uncooled sensors are increasingly paired with onboard Application-Specific Integrated Circuits (ASICs) that handle analog-to-digital conversion and noise filtering right on the silicon die itself. That level of integration is really what's opened thermal imaging up to everyone, it's now low-power and self-contained enough to fit into portable diagnostic tools, autonomous systems, and consumer electronics [10].

4 Theoretical Fundamentals

Building an autonomous, embedded thermal imaging system starts with a hard look at what hardware is available. Turning raw infrared telemetry into a standalone diagnostic tool means every component has to be chosen carefully, weighing processing power against energy efficiency, physical size, price, and how complicated it'll be to integrate.

This part walks through the theoretical groundwork behind the hardware the system needs. It works through a systematic study of the microprocessors, sensor arrays, power management topologies, and visual interfaces currently available on the market. By weighing what the project needs against what each competing solution offers, this part builds a clear case for every component that ends up in the final hardware architecture.

4.1 Central Processing Core

4.1.1 System Requirements

In any embedded optoelectronic system, the central processing unit sits at the core, coordinating the power management network, the visual interface, and the thermal sensor. Since this project is meant to become a physical, portable, autonomous thermal camera, choosing this central unit comes down to three critical engineering constraints:

- **Computational Performance:** The infrared sensor outputs raw telemetry as a low-resolution 32×24 spatial matrix, and the central unit has to upscale that in real time into a smooth 320×240 video feed. That takes real floating-point bandwidth and memory, enough to handle bilinear spatial interpolation on the fly without bottlenecking, on top of just running a headless operating system in the background.
- **Power Consumption and Thermal Dissipation:** Since this is a portable device running off a local DC battery, the processor has to stay within a tight thermal and electrical budget. Heavy computational loads naturally generate heat, and if that heat isn't kept in check, it'll bleed through the compact 3D-printed enclosure and throw off the infrared sensor's delicate temperature baseline sitting right next to it. The processor also needs to hold up against voltage droops during those heavy computational spikes.
- **Hardware Connectivity:** The unit needs native, hardware-level support for a specific set of protocols: a high-speed, bidirectional I2C bus for thermal telemetry, a one-way 4-wire SPI bus for driving the display, and standard GPIO pins for edge-triggered safety interrupts. On top of that, onboard wireless connectivity is necessary, since it's what allows remote, headless calibration without plugging in any external peripherals.

4.1.2 Analyzed Solutions

To meet the constraints laid out above, the computational core has to strike a careful balance between raw processing power, electrical efficiency, physical size, and cost. During the architectural design phase, three categories of processing units were put on the table: bare-metal microcontrollers, full-sized single-board computers (SBCs), and compact System-in-Package (SiP) microcomputers.

ESP32 Microcontroller.

Microcontrollers are a popular choice for portable embedded systems, and for good reason, they are cheap, extremely energy-efficient, boot instantly, and give you deterministic hardware control. Advanced options like the ESP32 even come with integrated 802.11 b/g/n Wi-Fi built in, which covers the system's remote-access requirement right out of the box, at a fraction of what a full computer would cost. The problem is floating-point bandwidth and RAM, there's just not much of either. Espressif's specifications list 520 KB of internal SRAM on a typical ESP32 [11].

That's plenty for capturing raw I2C telemetry off the thermal sensor, but the moment you try to buffer and upscale that data, you hit a wall. Interpolating a low-resolution array into a high-resolution 320×240 matrix means thousands of floating-point calculations per frame, and on a bare-metal microcontroller, that mathematical load translates into unacceptable latency and frame rates that drop off.

Raspberry Pi 4 Model B.

To get past the microcontroller's math bottleneck, high-performance SBCs were the next option considered. A full-sized board like the Raspberry Pi 4 Model B runs a multi-core 64-bit Cortex-A72 CPU backed by multiple gigabytes of high-speed LPDDR4 RAM. Computationally, this is more than enough, it can go through heavy matrix transformations in real time without problems. But that power comes with real costs. For one, full-sized SBCs are a much bigger hardware investment. They also need serious current, the official Raspberry Pi 4 datasheet calls for a minimum 3.0A power supply, which would drain a portable battery bank fast. On top of that, high processing speeds mean real heat, and inside a sealed 3D-printed enclosure, that heat has nowhere to go. It would inevitably bleed into the nearby far-infrared sensor and throw off the diagnostic readings. And finally, a desktop-class SBC is just too bulky to make the device genuinely portable [12].

Raspberry Pi Zero Series.

This is where the hybrid category comes in. An attempt to get the processing power of a full SBC without giving up the small footprint and low cost of a microcontroller. These boards use highly integrated System-in-Package (SiP) designs to pack in a 64-bit multi-core processor, enough SDRAM, and full Linux support, while deliberately keeping clock speeds down to save power. In theory, this gives exactly the processing bandwidth spatial interpolation needs, without sacrificing the thermal, electrical, or financial budget [13].

To summarize the architectural evaluation, Table 1 outlines the core operational metrics of the three analyzed processing categories.

Table 1 Comparison of Processing Architectures

Evaluation Criteria	ESP32	Raspberry Pi 4	Raspberry Pi Zero 2 W
Processor Architecture	32-bit Dual-Core	64-bit Quad-Core	64-bit Quad-Core
System RAM	520 KB SRAM	1 GB to 8 GB LPDDR4	512 MB LPDDR2
Power Consumption	Low (< 0.5A)	High ($\geq 3.0A$)	Moderate (< 1.5A)
Clock Frequency	240 MHz	1.5 GHz	1.0 GHz
Physical Dimensions	~55 mm × 28 mm	85 mm × 56 mm	65 mm × 30 mm
Price	5-10 USD	45-75 USD	15 USD

4.1.3 Adopted Solution

The Raspberry Pi Zero 2 W was ultimately adopted as the primary central processing core for this embedded optoelectronic architecture. Figure 6 shows the top-down physical layout of the adopted logic board. A few key engineering reasons drove this choice.

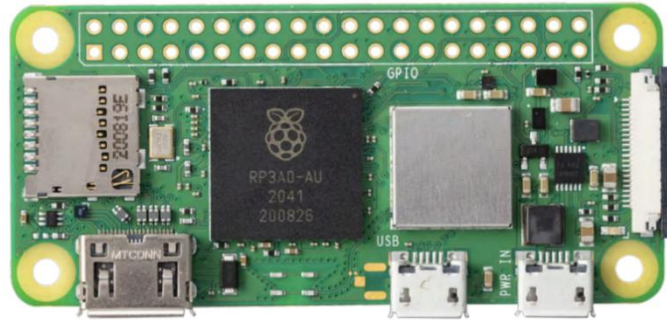


Figure 6. Top View Raspberry Pi Zero 2 W [13]

First, on the hardware side, the board is built around a highly integrated Raspberry Pi RP3A0 System-in-Package (SiP), which combines a Broadcom BCM2710A1 silicon die directly with 512MB of LPDDR2 SDRAM. It runs on a quad-core 64-bit Arm Cortex-A53 processor clocked at a stable 1.0 GHz, enough mathematical bandwidth and multi-threaded headroom to comfortably handle generic embedded applications and a range of algorithmic workloads. Unlike lightweight microcontrollers, which tend to hit memory limits fast under heavy operations, this board handles multiple execution threads smoothly while still keeping power draw low and thermal output well-managed. That matters here specifically because it means excess internal heat won't bleed into the adjacent infrared sensor array and throw off the environmental baseline readings [13].

Table 2 breaks down the core platform parameters in full.

Table 2 Technical Specifications

Evaluation Parameter	Specification Details
System Architecture	Broadcom BCM2710A1, Quad-core 64-bit SoC
Processor Core / Clock Speed	Arm Cortex-A53 1.0 GHz
System Memory	512MB LPDDR2 SDRAM
Integrated Wireless Connectivity	2.4 GHz IEEE 802.11b/g/n Wireless LAN, Bluetooth 4.2, BLE
Storage Interface	MicroSD Card Slot
Physical Dimensions	65 mm × 30 mm
Operating Temperature Range	-20°C to +70°C

Beyond raw computational throughput, this board also earns its place through a dense connection topology and a solid lineup of built-in peripheral ports. As Figure 6 shows, it fits an impressive array of industry-standard connectors into a very tight surface area. The standout feature is a standard 40-pin GPIO header, which gives the system native, hardware-level support for high-speed bidirectional I2C communication and 4-wire SPI data routing. Along the edges of the board sit several secondary connectors: a mini HDMI port for external video output, a USB 2.0 port with OTG capability for peripheral pairing, a dedicated micro USB socket for DC power input, and a CSI-2 camera connector [13].

Electrically speaking, running a full operating system inside a battery-powered, self-contained diagnostic enclosure means sticking closely to the board's power tolerances to avoid runtime brownouts. The manufacturer's documentation specifies an input requirement of 5V DC at 2.5A, enough overhead to handle heavier downstream USB peripheral loads, even though the board's own baseline draw stays efficient enough for portable use. Table 3 lays out the verified power parameters and interface components [13].

Table 3 Electrical and Power Input Parameters

Power Parameter	Operational Threshold / Requirement
Input Power	5V DC 2.5A
Primary Power Ingestion Interface	Micro USB Socket
Peripheral Power Bus	1 × USB 2.0 interface with OTG
External Display Interface	Mini HDMI Port

Finally, the board's small form factor and built-in wireless capability line up well with the mechanical goals of the system. Because the entire computing platform fits within such tight physical boundaries, it makes it possible to build a genuinely compact, ergonomic handheld enclosure. Figure 7 shows the exact mechanical layout and dimensions provided by the manufacturer, which in turn dictate where the internal mounting points go, how much clearance the connector ports need, and the outer dimensional limits of the final 3D-printed frame.

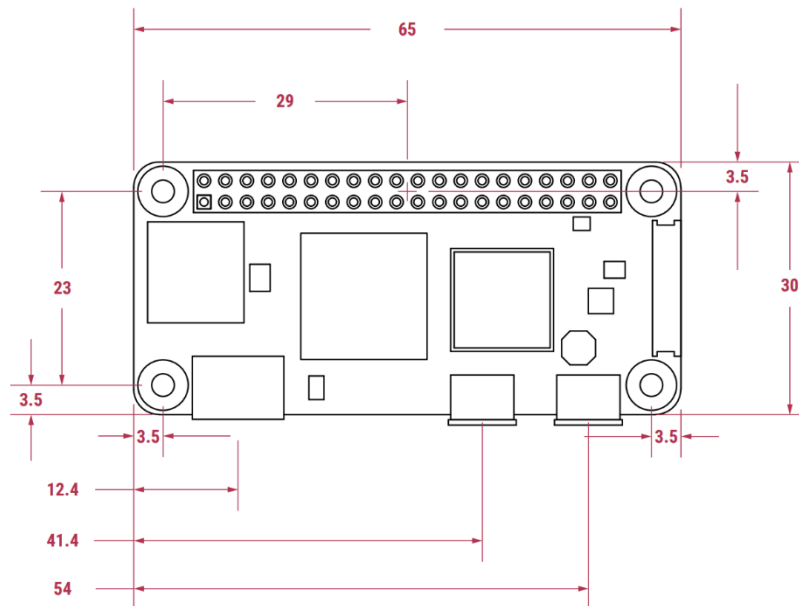


Figure 7. Mechanical Dimensional Blueprint and Spatial Layout [13]

It's also worth noting that since the finished device runs as a fully headless, standalone module having onboard wireless built in turns out to be a real development advantage. It allows secure remote SSH access over a network connection, so a developer can check logs, verify code stability, and push firmware updates directly to the board without ever having to crack open the 3D-printed case or plug in external data lines.

Taken together, solid quad-core processing power, a small mechanical footprint, native hardware communication support, and built-in wireless administration the Raspberry Pi Zero 2 W stood out as the clear choice for this architecture. It meets every operational requirement the project set out without the thermal load or power overhead that comes with desktop-class boards, making it the right computational core for a handheld thermal diagnostic tool.

4.2 Infrared Thermal Sensor

4.2.1 Principles of Infrared Thermography and Sensor Technologies

For the embedded optoelectronic architecture to diagnose anything, it needs a component capable of "seeing" heat. Infrared thermography rests on a simple physical fact: any object above absolute zero gives off electromagnetic radiation. How much of that radiation it emits comes down mainly to temperature the hotter the object, the more infrared energy it puts out. For objects at everyday terrestrial and biological temperatures, that emission peaks in the long-wave infrared (LWIR) band, roughly $8\text{ }\mu\text{m}$ to $15\text{ }\mu\text{m}$. Where this band sits relative to visible light and microwave frequencies, along with its various sub-bands, is shown in Figure 8 [14].

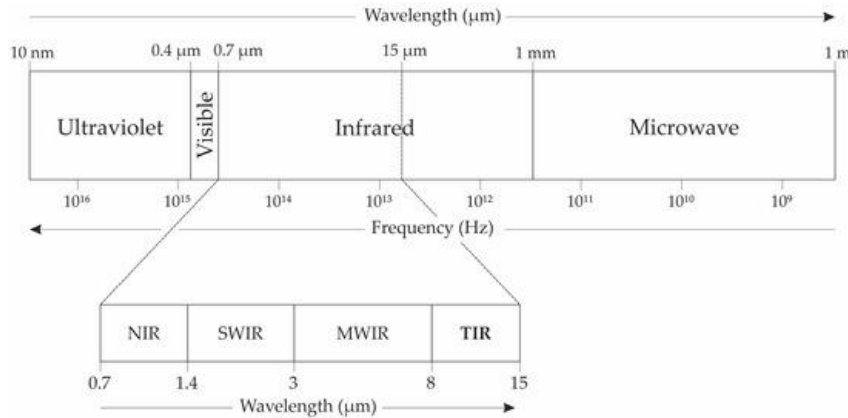


Figure 8. The Electromagnetic Spectrum, Highlighting the Conventional Infrared Sub-Bands [14]

The total power a black body radiates follows the Stefan-Boltzmann Law:

$$P = \epsilon \sigma A T^4 \quad (3)$$

where:

P — the total radiated power [W];

ϵ — the emissivity of the object's surface (dimensionless, $0 \leq \epsilon \leq 1$);

σ — the Stefan-Boltzmann constant, approximately $5.67 \times 10^{-8} \text{ W}/(\text{m}^2 \cdot \text{K}^4)$;

A — the surface area of the emitting object [m^2];

T — the absolute temperature of the target object [K].

An infrared thermal sensor's whole job is to catch this invisible emitted power and turn it into a measurable electrical signal, all without touching the target [5].

Infrared sensing technologies generally fall into two categories, depending on how they physically operate: Photon (Quantum) Detectors and Thermal Detectors.

Quantum detectors work by absorbing infrared photons directly into the sensor's semiconductor lattice, producing a fast, highly sensitive electrical response. They offer excellent resolution and frame rates, but they come with a serious catch: they need cryogenic cooling to get rid of thermally induced dark current, essentially, electronic noise generated by the sensor's own room-temperature heat. That need for bulky, power-hungry cooling keeps quantum detectors confined to military aerospace, astronomy, and high-end industrial labs, which rules them out entirely for a portable, battery-powered embedded system [15].

Thermal detectors work differently. Rather than measuring photon-electron interactions, they absorb infrared radiation and let it physically change the temperature of the sensing material itself. That temperature shift then alters some measurable electrical property: resistance, or

generated voltage. Because they depend on an actual heating-and-cooling cycle, thermal detectors respond more slowly than quantum detectors do, but in exchange, they can operate at room temperature with no cooling at all. That tradeoff is exactly what makes them the practical choice for compact, low-power optoelectronic devices [15].

Within uncooled thermal detectors, there are three main technologies worth distinguishing.

Pyroelectric sensors rely on materials that spike a temporary voltage whenever their internal temperature changes, since they only respond to fluctuations, not absolute values, they're commonly used for basic motion detection but can't measure a static temperature [16].

Microbolometers, on the other hand, use a grid of tiny resistors whose resistance drops as infrared radiation heats them. They can produce very high-resolution thermal images, but they tend to be expensive, need complex calibration, and often require a mechanical shutter to periodically recalibrate against ambient drift [16].

Finally, thermopile arrays work off the Seebeck effect, a series of thermocouples that generate a continuous, proportional voltage relative to a cold reference junction when infrared radiation heats their active junction. Unlike pyroelectric sensors, thermopiles can measure absolute static temperatures. Advances in CMOS and MEMS fabrication have made it possible to pack arrays of these thermopiles onto a single silicon chip, the standard cross-sectional structure of one of these uncooled pixels is shown in Figure 9. Thermopile arrays don't match the resolution of high-end microbolometers, but they're considerably cheaper, sip very little power, and need no moving shutter at all, which makes them a natural fit for portable embedded devices [16].

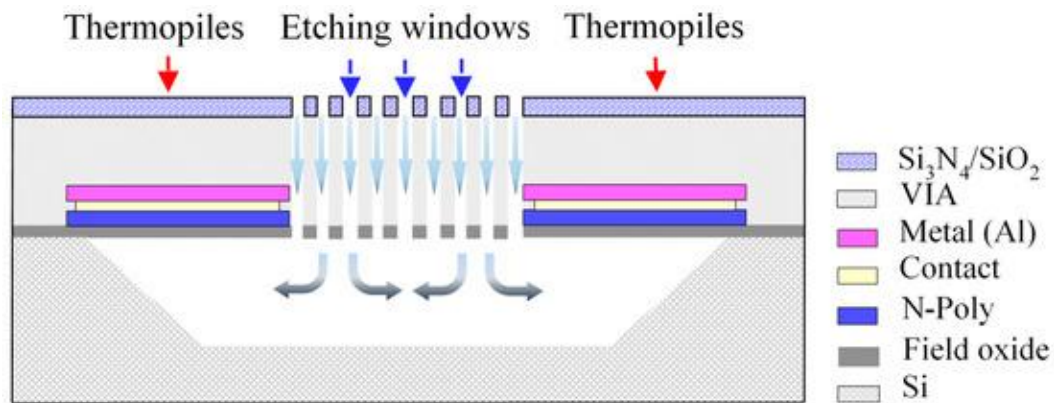


Figure 9. Schematic Drawing of the Cross-Section of a CMOS-MEMS Thermopile Architecture [16]

4.2.2 Analyzed solutions

To figure out the best option for spatial temperature measurement within this hardware setup, it's worth comparing the leading commercial technologies directly. The primary infrared sensor needs to balance spatial resolution, power draw, structural complexity, and cost. Three thermal imaging modules tend to come up as the industry benchmarks across different performance tiers: the Panasonic AMG8833 (low-tier thermopile array), the Melexis MLX90640 (mid-tier thermopile array), and the FLIR Lepton 3.5 (high-tier microbolometer). Table 4 lays out how these three compare on key parameters, architecture, and interface protocols [17].

Table 4 Technical Comparison of Thermal Imaging Sensors

Evaluation Parameter	Panasonic AMG8833	Melexis MLX90640	FLIR Lepton 3.5
Sensor Technology	Thermopile Array	Thermopile Array	VOx Microbolometer
Spatial Resolution	8 × 8 (64 pixels)	32 × 24 (768 pixels)	160 × 120 (19,200

			pixels)
Data Interface	I2C	I2C	SPI / I2C (CCI)
Maximum Refresh Rate	10 Hz	64 Hz	8.7 Hz
Ambient Operating Range	0°C to +80°C	-40°C to +85°C	-10°C to +80°C
Target Measurement Range	-20°C to +100°C	-40°C to +300°C	-10°C to +400°C
Mechanical Shutter	None (Shutterless)	None (Shutterless)	Integral Mechanical Shutter

Comparing the two solid-state thermopile arrays against each other, the differences really come down to spatial fidelity and thermal dynamics. Both the Panasonic AMG8833 and the Melexis MLX90640 share the same basic CMOS architecture, talk over standard I2C, and have no moving parts at all [18][19].

The AMG8833 is held back by its 8×8 pixel grid, 64 data points is enough for basic presence detection, but the pixels are large enough that distinguishing clean thermal boundaries gets noisy fast. It doesn't offer the fine detail needed for something like spotting localized overheating on a circuit board. The MLX90640, by contrast, delivers a 32×24 matrix: 768 individual temperature readings per frame. It keeps the same simple I2C setup as the AMG8833, but pulling usable thermal data out of it means the host processor has to run floating-point calculations against factory EEPROM parameters, something the Raspberry Pi Zero 2 W handles without issue. That twelvefold jump in data points gives the resolution needed for real thermographic analysis, without dragging in the added complexity of a microbolometer [18][20].

At the top end, the FLIR Lepton 3.5 offers a resolution nobody else here can touch: 160×120 pixels, built around a Vanadium Oxide (VOx) microbolometer core. Its image clarity and wide measurement range, up to +400°C, are well beyond anything a thermopile array can manage, but that comes at a real engineering cost for a lightweight project. For one, being a microbolometer means it needs an internal mechanical shutter, commonly called Flat Field Correction (FFC), that periodically closes to recalibrate against ambient drift. That's a mechanical failure point, and it causes brief freezes in the video feed. It also means handling a 19,200-pixel array over a high-speed SPI bus, which complicates the software drivers considerably, demands DMA controllers, and eats up a large chunk of the host processor's available memory. And on top of all that, the vacuum packaging and micromachining needed to build microbolometers make them expensive, too expensive for a low-cost, accessible diagnostic tool [21][22].

Weighing all these trade-offs together, the Melexis MLX90640 comes out as the clear fit for this architecture. It sits right in the middle, well past the limitations of a basic motion sensor, but without the cost and complexity of an industrial-grade microbolometer, giving a solid balance of cost, diagnostic performance, and simplicity. It's a fully solid-state, shutterless thermal array that runs cleanly over a standard data bus, offering enough spatial resolution for real thermographic work without overloading the processing bandwidth, power budget, or cost constraints of a portable device [18][20].

4.2.3 Adopted solution

With the Melexis MLX90640 settled on as the thermal imaging core, it's worth digging into its internal architecture and specs to understand how it actually fits into the broader circuit. Physically, the sensor sits in an industry-standard, 4-pin TO-39 metal can package. That hermetically sealed enclosure does double duty: it protects the delicate silicon inside from

physical damage and moisture, and it also acts as a thermal mass that helps stabilize the sensor's internal ambient temperature, which matters a lot for accurate thermopile baseline readings [18].

Internally, the MLX90640 is a tightly integrated System-in-Package (SiP). At the front end, shown in the block diagram in Figure 10, sits the 32×24 infrared-sensitive thermopile array.

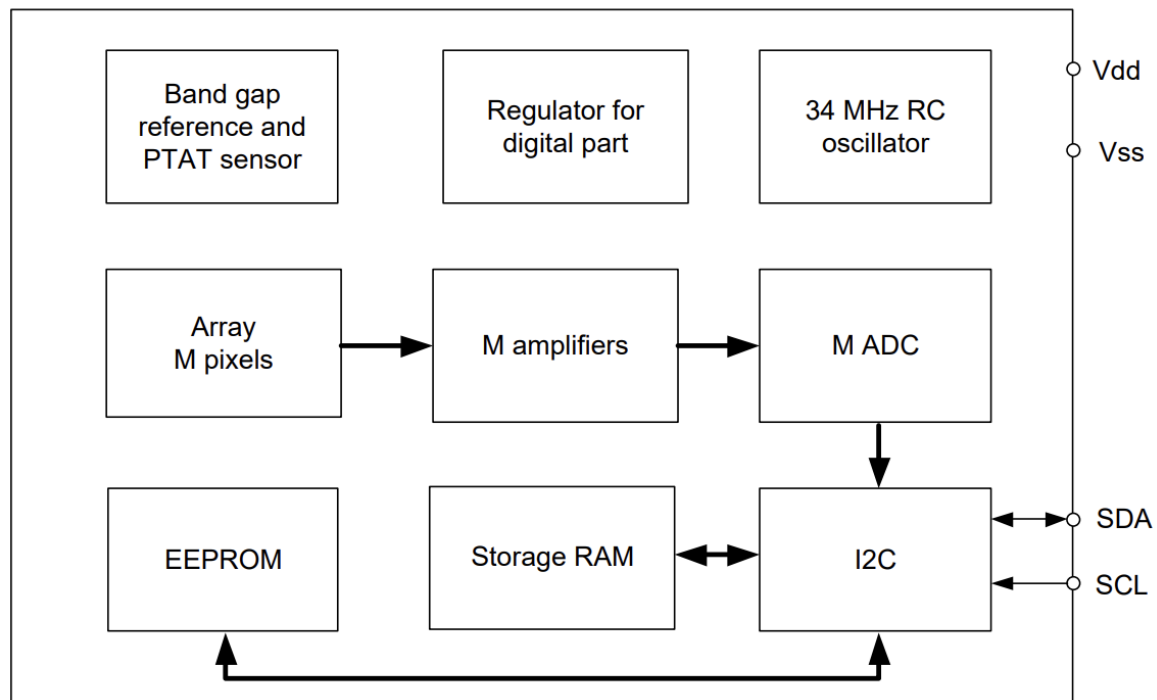


Figure 10. Internal Hardware Block Diagram of the Melexis MLX90640 Sensor [18]

When infrared radiation hits these elements, the resulting microvolt-level signals get routed through an internal multiplexer into a programmable-gain low-noise amplifier (LNA). Since these thermal voltages are tiny and easily disturbed by electromagnetic interference, they're digitized right away on-chip by a programmable high-precision Analog-to-Digital Converter (configurable up to 19-bit). From there, the raw pixel data gets buffered into the sensor's internal RAM, where the host processor can pull it via the I²C bus. That I²C interface runs in Fast Mode Plus at up to 1 MHz, which gives it enough bandwidth to extract full thermal frames at refresh rates up to 64 Hz [18].

One thing that really defines the MLX90640's architecture is how it handles memory and data extraction. Unlike simpler sensors that hand you a finished temperature reading, the MLX90640 only stores raw ADC measurements in RAM. Turning that into an actual Celsius temperature means the host processor has to run a fairly involved sequence of floating-point calculations. The calibration constants those equations depend on things like individual pixel sensitivity, offset thermal gradients, and ambient drift coefficients, are burned permanently into the sensor's EEPROM at the factory. So at startup, the microcomputer has to pull down the entire EEPROM payload, store it locally, and keep running those calibration algorithms against the incoming RAM data continuously. In effect, this pushes most of the math off the sensor and onto the Raspberry Pi Zero 2 W [18].

There's also a subpage scanning mechanism involved in how the sensor reads out its 768 pixels, it doesn't grab the whole array at once. Instead, the array is split into two subpages (Subpage 0 and Subpage 1), which get read in either an alternating chess (checkerboard) pattern or an interleaved row pattern, depending on how the sensor's configured. Figure 11 shows the standard chess pattern.

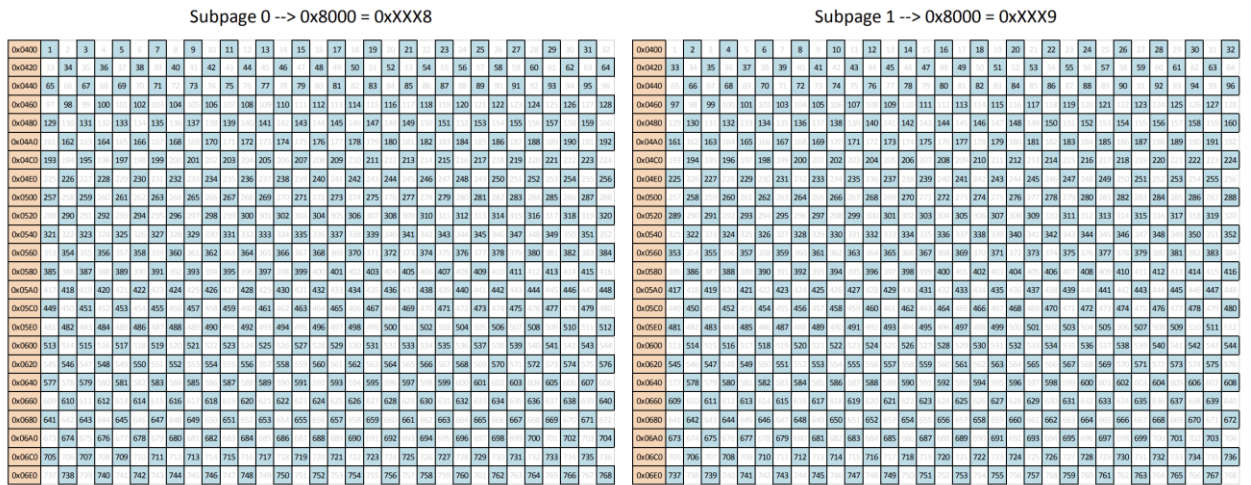


Figure 11. Subpage Alternating Chess Pattern Pixel Reading Configuration [18]

The host processor then has to stitch these two subpages back together in software to get a clean, artifact-free thermal image. Optically, the sensor uses a specialized silicon lens built directly into the TO-39 package lid. The MLX90640 comes in two field-of-view options: a standard $55^\circ \times 35^\circ$ and an ultra-wide $110^\circ \times 75^\circ$ and this project deliberately went with the narrower 55° version. Packing the same 768 pixels into a tighter optical angle boosts spatial resolution at a given distance compared to the wide-angle variant. That's exactly what matters for a diagnostic tool meant to inspect localized thermal anomalies, things like PCB diagnostics or targeted temperature readings, where getting more pixels focused on a specific point matters more than covering a wide field of view [18].

4.3 Display

In any diagnostic instrument, acquired sensory data must be translated into a comprehensible format through a visual interface. For a portable thermal imaging device, this requires rendering two-dimensional, color-mapped spatial data within strict electrical and computational constraints, which precludes the use of simple segmented displays and necessitates an active-matrix Thin-Film Transistor Liquid Crystal Display (TFT-LCD).

Unlike emissive OLED displays, an LCD depends on a constant LED backlight. Light passes through a rear polarizer, a layer of liquid crystal molecules, and red, green, and blue sub-pixel color filters. At each pixel junction, a microscopic silicon transistor applies a precise voltage to alter the orientation of the liquid crystal, functioning as an analog light shutter [24].

Standard Twisted Nematic (TN) panels exhibit significant color inversion at off-axis viewing angles, a limitation that compromises the accurate interpretation of thermal color palettes. For this reason, In-Plane Switching (IPS) technology was selected instead. In an IPS panel, the liquid crystal molecules remain aligned parallel to the glass substrates and rotate exclusively within that plane when an electric field is applied. This planar rotation yields substantially improved color accuracy and wider viewing angles, avoiding the optical degradation characteristic of conventional TN configurations [25].

Driving a 2.0-inch, 240×320 resolution display directly requires continuously managing 230,400 individual RGB transistors (76,800 pixels $\times$ 3 colors), a demand that would exceed the general-purpose input/output (GPIO) capacity of a standard microcomputer. To address this constraint, the display glass is bonded directly to a dedicated controller: the Sitronix ST7789 [23].

As shown in Figure 12, the ST7789 functions as an intermediary hardware translator. It incorporates an internal Graphics Random Access Memory (GRAM) buffer capable of storing a complete frame in 16-bit RGB565 color format, allowing the host processor to transmit calculated pixel color data over a high-speed Serial Peripheral Interface (SPI) bus. The controller

then independently generates the row and column timing voltages required to refresh the TFT matrix at 60 Hz. By offloading this multiplexing task to the ST7789, the Raspberry Pi Zero 2 W is able to direct its processing resources primarily toward acquiring and interpreting data from the MLX90640 thermal sensor [23].

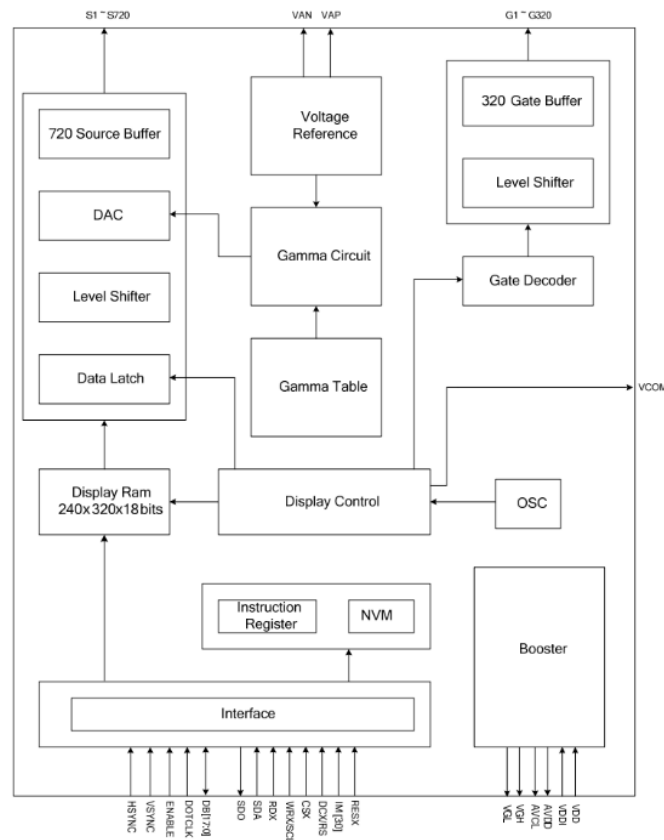


Figure 12. Internal Hardware Block Diagram of the ST7789 Display Controller [23]

4.4 Communication Protocols

In modern embedded systems, central processing units rely on synchronous serial communication buses to exchange digital data with peripheral components. These protocols transmit data sequentially, bit by bit, maximizing data throughput while minimizing the physical space required for circuit board traces.

4.4.1 Inter-Integrated Circuit (I²C)

The Inter-Integrated Circuit (I²C) protocol is a synchronous, multi-master, half-duplex serial communication bus designed for short-distance data exchange. Its principal advantage lies in physical simplicity: regardless of how many peripherals are connected, the bus requires only two active wires: a Serial Data line (SDA) and a Serial Clock line (SCL) [26].

Electrically, I²C employs an open-drain architecture. Devices on the bus can actively pull the signal lines to a logic low (0V), but return to a logic high (e.g., 3.3V) only through external pull-up resistors when idle. This arrangement prevents short circuits if multiple devices attempt to communicate simultaneously. Because all devices share the same two wires, I²C dispenses entirely with individual hardware selection pins, instead relying on a software-based addressing scheme. The master device initiates communication by broadcasting a unique 7-bit or 10-bit

hexadecimal address alongside the clock signal; only the peripheral whose address matches (a thermal sensor, for instance) responds, allowing efficient data transfer over a minimal physical footprint [26][27].

4.4.2 Serial Peripheral Interface (SPI)

The Serial Peripheral Interface (SPI) is a synchronous, full-duplex, single-master serial bus engineered specifically for high-bandwidth data streaming. It is widely used in embedded architectures to interface with components requiring rapid, continuous data transfer, such as high-resolution displays or external memory modules [28].

A standard SPI configuration operates over four dedicated physical lines: a Serial Clock (SCLK), generated by the master processor to synchronize timing; a Master Out Slave In (MOSI) line for transmitting data to the peripheral; a Master In Slave Out (MISO) line for receiving data from the peripheral; and a Chip Select (CS) line. Electrically, SPI employs a push-pull architecture, in which devices actively drive the voltage signal both high and low. This more robust signaling method allows SPI buses to reach considerably higher clock frequencies, enabling substantially greater data throughput than I²C [27].

Communication on an SPI bus is initiated through a direct hardware trigger: the master processor selects a given peripheral by pulling its dedicated Chip Select (CS) pin to a logic low. This hardware-level activation establishes an immediate, dedicated connection without the software-based addressing overhead required by I²C. Because data flows simultaneously in both directions along the MISO and MOSI lines, the protocol is particularly well suited to sustained, uninterrupted data streams [28].

4.5 Power Management and Energy Storage

The integration of complex microprocessors, active-matrix display controllers, and high-resolution thermal sensors into a portable form factor imposes strict constraints on the system's power architecture. As the device must operate independently of grid power, a localized energy storage solution is required that can sustain high current draw while maintaining an exceptionally stable voltage supply.

4.5.1 Lithium-Ion Battery

To meet the high energy density and low spatial footprint demanded by modern portable instruments, embedded systems predominantly rely on Lithium-ion (Li-ion) battery chemistry. Unlike traditional nickel-cadmium (NiCd) or lead-acid cells, Li-ion batteries operate through the movement of lithium ions from the negative electrode (anode) to the positive electrode (cathode) during discharge, and in the reverse direction during charging [29].

This chemical process yields a favorable energy-to-weight ratio, which has made Li-ion the industry standard for mobile electronics. A standard single-cell Li-ion battery provides a nominal voltage of 3.7V, though this value is not fixed; rather, it follows a predictable discharge curve. At full charge, the cell reaches a peak of 4.2V, and as the system draws on this stored energy, the voltage steadily declines toward a safe cutoff threshold of approximately 3.0V. Accounting for this discharge curve is essential in embedded design, since the raw battery voltage typically must be regulated to supply the stable 5.0V or 3.3V logic levels required by the Raspberry Pi and its peripheral sensors [29][30].

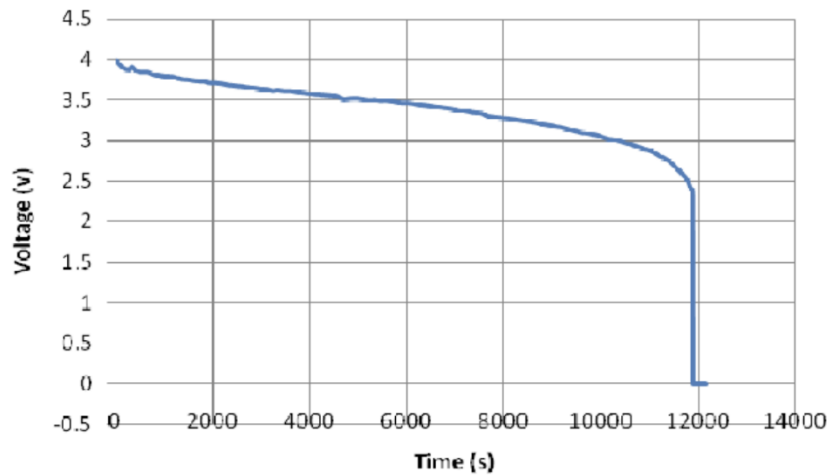


Figure 13. Experimental discharge voltage curve of a Lithium-ion cell over time, demonstrating the gradual voltage drop and sudden protection cutoff [31]

4.6 Battery Management Systems (BMS)

Despite its high energy density, Li-ion chemistry is inherently volatile. Operating outside strict voltage and current thresholds leaves lithium cells vulnerable to permanent chemical degradation, swelling, or catastrophic thermal runaway. For this reason, a raw Li-ion cell must never be connected directly to a microprocessor load without an intermediary supervisory circuit: a Battery Management System (BMS) which continuously monitors the cell's real-time state and physically disconnects the load if unsafe conditions arise [30].

A fundamental BMS architecture comprises three protective mechanisms operating in tandem. The first is overcharge protection, achieved by monitoring the charging state and severing the input circuit once the cell reaches its maximum threshold, typically 4.2V; this prevents metallic lithium plating and thermal breakdown that would otherwise result from forcing current into an already saturated cell. The second is over-discharge protection: allowing a Li-ion cell to drop below 3.0V causes the internal copper current collectors to dissolve, permanently reducing the battery's capacity, so the BMS automatically cuts power to the entire embedded system once this lower threshold is reached. The third mechanism addresses over-current and short-circuit conditions, using internal shunt resistors to monitor outgoing current; should a hardware fault produce a sudden, large current spike, the BMS triggers onboard field-effect transistors (FETs) to interrupt the circuit within milliseconds, preventing overheating. Through the coordinated management of these three parameters, the BMS ensures both the longevity of the Li-ion cell and the physical safety of the diagnostic instrument [30].

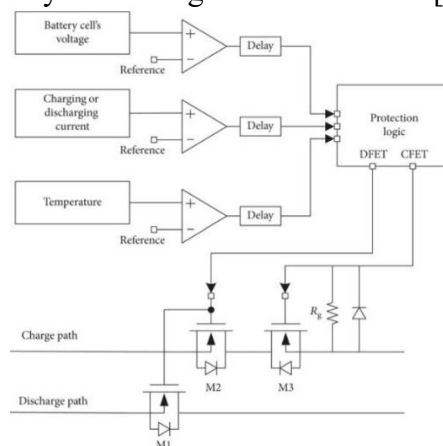


Figure 14. Internal block diagram of a Li-ion Battery Management System (BMS) [32]

5 Implementation

This implementation phase marks the point at which the proposed theoretical architecture is translated into a functional, autonomous embedded system. The central aim is to physically realize a portable, "plug-and-play" thermal camera capable of operating entirely independently of external peripherals.

At the heart of this design lies the hardware integration, built around the Raspberry Pi Zero 2 W as the core processing unit. This single-board computer interfaces directly with the MLX90640-BAB infrared array via the I2C protocol, through which it acquires raw thermal telemetry, while real-time visual output is handled by a 2.0-inch IPS display driven by an integrated ST7789 controller. To sustain this compact architecture, a custom power delivery network draws on two 18650 3.7V Li-ion cells connected in parallel, with a dedicated Battery Management System (BMS) regulating the supply. Running on a headless operating system, these components together form a responsive, standalone diagnostic tool.

Equally central to the hardware assembly is the software architecture that binds these peripheral components together. The system's core logic was developed entirely in Python, chosen for its robust library ecosystem and its seamless integration with embedded Linux hardware. The software pipeline was designed to manage high-frequency data ingestion and the associated matrix mathematics without overwhelming the processor: NumPy handles rapid floating-point array calculations, while Matplotlib applies the thermal color gradients, together allowing the system to process the sensor's 768-pixel resolution in real time. The low-level hardware communication: I2C data polling, SPI display rendering, and GPIO state monitoring for safe shutdowns. They were implemented using the Adafruit Blinka hardware abstraction layer.

5.1 Block diagram

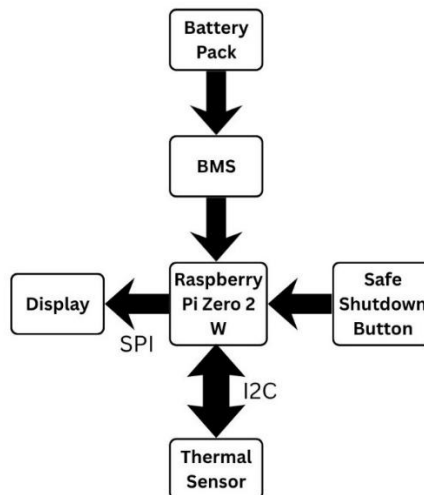


Figure 15. Block diagram

Figure 15 details the hardware architecture and signal routing of the thermal camera. Power flows from the parallel battery pack through the BMS, which supplies a regulated 5V direct

current to the Raspberry Pi Zero 2 W. This single-board computer (SBC) acts as the central processing hub, managing three dedicated data interfaces: a bidirectional I2C bus for acquiring raw telemetry from the thermal sensor, a unidirectional 4-wire SPI bus for transmitting processed image frames to the display module, and a GPIO input connected to an ON-OFF switch that triggers a safe operating system shutdown prior to power loss.

5.2 Central Processing Unit: Raspberry Pi Zero 2 W

The central processing core selected for this embedded architecture is the Raspberry Pi Zero 2 W. Unlike traditional bare-metal microcontrollers, this device is a highly integrated single-board computer (SBC) capable of running a complete, headless Linux operating system. At its heart is the Raspberry Pi RP3A0 System-in-Package (SiP), which integrates a Broadcom BCM2710A1 die featuring a quad-core 64-bit ARM Cortex-A53 processor clocked at 1.0 GHz, alongside 512MB of LPDDR2 synchronous dynamic random-access memory (SDRAM) [13].

This particular SBC was chosen because it occupies something of a computational "sweet spot" for a portable thermal imaging system. Standard microcontrollers often lack the floating-point processing bandwidth needed for continuous, real-time matrix transformations, such as the bilinear interpolation required to upscale the raw 32×24 thermal array into a fluid 240×320 RGB video feed. The Raspberry Pi Zero 2 W handles these demanding Python-based array manipulations without any real architectural bottleneck, all within an ultra-compact footprint of just 65 mm by 30 mm. It manages this throughput while keeping power consumption low, which matters here since excess CPU heat could otherwise interfere with the adjacent infrared sensor's readings [13].

Beyond raw computational power, the board's peripheral and connectivity options align well with the system's requirements. Its standard 40-pin GPIO header offers native, hardware-level support for the communication protocols central to this project: the high-speed I2C bus for sensor telemetry, the 4-wire SPI bus for display rendering, and standard logic pins for the physical safe-shutdown interrupt. Finally, its integrated 802.11 b/g/n Wi-Fi proved to be a valuable development asset in its own right. Since the finished system runs as a headless, autonomous unit with no graphical desktop, this onboard wireless connection allows for remote SSH access, making it possible to calibrate code, monitor the system, and push script updates over the network, without ever needing to physically open the device or attach external peripherals [13].



Figure 16. Raspberry Pi Zero 2 W [13]

5.2.1 Hardware Interconnections and Pin Mapping

To ensure signal integrity and reliable operation, the physical connections between the Raspberry Pi Zero 2 W and its peripheral modules were mapped according to strict logic-level requirements. The following hardware routing was implemented for the central SBC setup.

For temperature sensing, the MLX90640 communicates via the high-speed I2C protocol. Although some breakout boards for this sensor include an onboard low-dropout (LDO) regulator to permit 5V operation, the module in this system is powered directly from the Raspberry Pi's 3.3V rail instead. Bypassing the LDO keeps the sensor's data lines natively aligned with the microcontroller's 3.3V logic, which eliminates the need for bidirectional logic level shifters altogether. The specific connections are routed as follows:

- Power Supply (VIN/VDD): Connected directly to the 3.3V rail (Physical Pin 1) of the Raspberry Pi.
- Common Ground (GND): Tied to system ground at GND (Physical Pin 6).
- Serial Data (SDA): Routed to the microcontroller's primary I2C data line at GPIO 2 (Physical Pin 3).
- Serial Clock (SCL): Routed to the microcontroller's primary I2C clock line at GPIO 3 (Physical Pin 5).

Similarly, dedicated connections were established to drive the visual interface, which uses a 2.0-inch LCD display with an integrated ST7789 controller. To sustain the data throughput needed for a fluid thermal video feed without graphical stuttering, the display is interfaced to the Raspberry Pi's primary SPI (SPI0) bus. This 4-wire configuration was wired as follows:

- Logic Power (VCC): Supplied by the secondary 3.3V rail (Physical Pin 17).
- Common Ground (GND): Tied to system ground at GND (Physical Pin 20).
- SPI Clock (SCL/SCK): Synchronized to the processor's clock output at GPIO 11 (Physical Pin 23).
- Master-Out-Slave-In (SDA/MOSI): Connected to the processor's data transmission line at GPIO 10 (Physical Pin 19).
- Chip Select (CS/CE0): Governed by an active-low signal on GPIO 8 (Physical Pin 24) to enable frame reception.
- Data/Command (DC): Controlled by GPIO 24 (Physical Pin 18) to differentiate between pixel data and configuration commands.
- Hardware Reset (RST/RES): Linked to GPIO 25 (Physical Pin 22) for strict initialization control.

Finally, to guard against file system corruption from abrupt power loss, a hardware-level safe-shutdown mechanism was implemented. A physical two-position toggle switch is wired directly between GPIO 21 (Physical Pin 40) and ground. The SBC's software configures this pin with an internal pull-up resistor, holding it at a stable HIGH state by default; engaging the switch pulls it LOW. Rather than continuously polling this pin, which would eat into the CPU cycles needed for real-time thermal matrix transformations, the background Python process instead relies on edge-triggered interrupts to catch this transition. As soon as a falling edge (HIGH to LOW) is detected, the system immediately executes a controlled shutdown, ensuring all operations halt safely before battery power is disconnected.

5.3 Thermal Sensor: MLX90640BAB

5.3.1 Operating Principles and Field of View

The visual core of the thermal imaging system is the MLX90640, a fully calibrated 32×24 pixel far-infrared (FIR) thermal sensor array. Unlike standard optical cameras, which capture visible light photons, this sensor works on the principle of thermoelectricity, detecting thermal radiation in the far-infrared spectrum. The physical array consists of 768 individual microscopic thermopile sensors: when infrared radiation from a target object strikes the active junction of a

thermopile, it absorbs the heat and generates a tiny, proportional voltage difference relative to a reference junction via the Seebeck effect [18].

This continuous change in analog voltage is the foundational metric the system relies on. By measuring these minute thermoelectric variations across the array, the sensor provides the raw differential data needed to calculate the absolute temperature of the external environment.

For this prototype, the BAB variant of the MLX90640 was chosen. According to the manufacturer's optical specifications, the BAB package is built with a narrower, more focused field of view (FOV), 55° on the horizontal axis and 35° on the vertical axis. Compared to the wider 110° "BAA" variant, this 55° FOV offers better spatial resolution and thermal targeting at medium distances, which makes it well suited to a handheld, directional thermal camera. Figure 17 illustrates this optical geometry, showing the exact spatial constraints of the elliptical cone captured by the sensor [18].

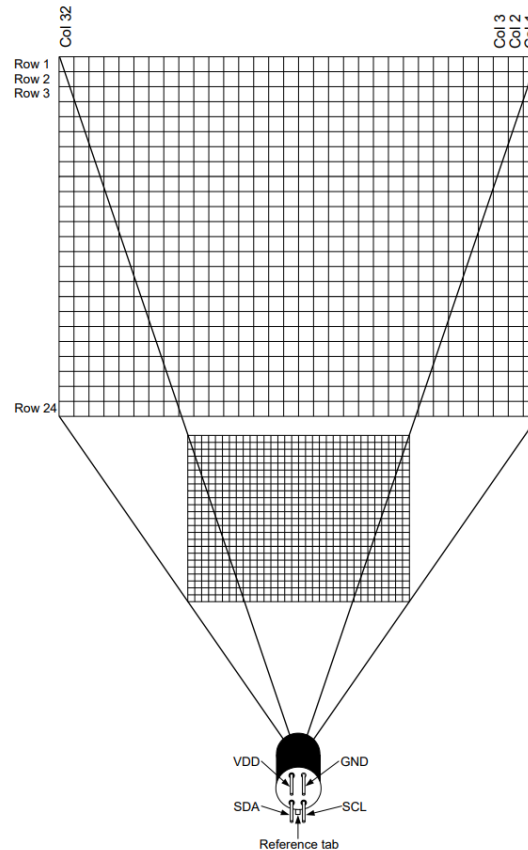


Figure 17. The geometric $55^\circ \times 35^\circ$ Field of View profile for the MLX90640-BAB [18]

5.3.2 Signal Acquisition and Electrical Characteristics

Before a complete digital data matrix can be sent to the host, the raw analog voltages generated by the thermopiles first need to be digitized at the silicon level. The MLX90640's internal architecture pairs a low-noise amplifier with a high-resolution 16-bit ADC. Since all 768 pixels can't be processed simultaneously, the chip divides the array into two interleaved subpages, reading the sensors in a staggered checkerboard pattern before storing the data in internal RAM [18].

To counteract microscopic manufacturing variations between pixels, the sensor also relies on a factory-programmed EEPROM containing large arrays of unique calibration constants. Electrically, it achieves a Noise Equivalent Temperature Difference (NETD) of 0.1K RMS and supports programmable refresh rates from a low-power 0.5 Hz up to a fluid 64 Hz. The internal routing needed to manage this signal path from the raw analog thermopiles, through the ADC,

and into the memory blocks is detailed in the manufacturer's architectural schematic in Figure 18 [18].

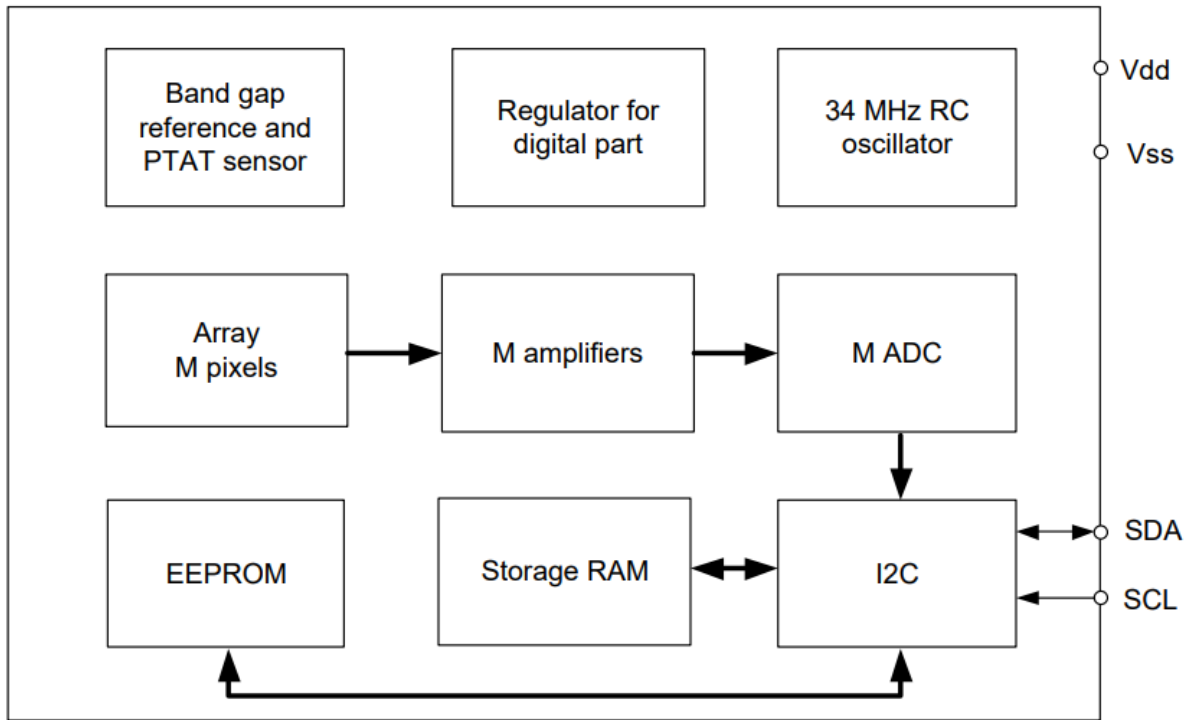


Figure 18. Internal architectural block diagram of the MLX90640 [18]

5.3.3 System Implementation

In the physical prototype, the MLX90640 functions as a digital slave device to the Raspberry Pi Zero 2 W over the I2C protocol, requiring only four connections:

- 3.3V power
- common ground,
- SDA
- SCL

Because retrieving the interleaved RAM frames and cross-referencing them against the EEPROM constants demands significant data bandwidth, the SBC's primary I2C bus was reconfigured to run at 1 MHz's. This higher clock rate ensures the full 768-pixel telemetry matrix is retrieved without bottlenecks, making it immediately available for Python-based interpolation. The simplicity of this high-speed integration, using only power and data lines, is shown in the module's pinout configuration in Figure Z [18].

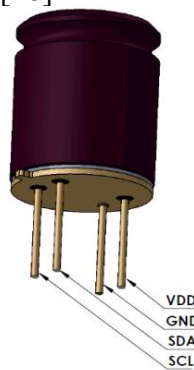


Figure 19. Physical 4-pin I2C configuration for the MLX90640 module [18]

5.4 Power Management and Energy Storage

To meet the design requirement of autonomous portability, the thermal camera is powered by a dedicated DC power subsystem. This subsystem handles three tasks: safely storing electrical energy, regulating the charging cycle, and delivering a stable voltage rail to the single-board computer and its peripherals. The architecture centers on IP5306, a highly integrated, multi-functional power management IC [33].

5.4.1 Lithium-Ion Energy Storage and USB-C Charging

The primary energy store consists of two 18650 cylindrical Li-Ion cells connected in parallel, giving a combined capacity of 4000mAh at a nominal 3.7V. These cells are replenished through a USB-C charging interface, with the IP5306 controller driving a synchronous switching topology that reaches charging efficiencies of up to 97%. The system automatically adjusts its charging current to match whatever adapter is connected, drawing up to 2.1A when rapidly recharging the parallel cells [33].

5.4.2 Step-Up Conversion and Line Drop Compensation

The battery bank's output is inherently volatile, ranging from 3.0V to 4.2V, yet the Raspberry Pi Zero 2 W requires a tightly regulated logic-level input to operate reliably. To bridge this gap, the IP5306 includes an integrated DC-DC synchronous boost converter switching at 500 kHz, which dynamically steps up the battery's variable output to deliver a continuous current of up to 2.4A at 91% conversion efficiency [33].

Notably, the module is designed with output line voltage compensation: rather than holding a flat 5.0V, it intentionally drives the baseline output to roughly 5.15V under load. This 0.15V offset proves particularly useful for Raspberry Pi architectures. Under heavy computational loads, such as the CPU spikes triggered by bilinear interpolation, transient voltage droops are unavoidable across the internal copper traces and polyfuses. By starting from 5.15V rather than 5.0V, the power delivery network ensures the voltage reaching the SoC stays above the critical 4.75V undervoltage lockout threshold, which in turn prevents CPU thermal throttling and system instability [33].

5.4.3 Power Path Management and Brownout Mitigation

One notable feature of the IP5306 is its built-in power path management, which supports charge-while-discharge operation. In principle, this pass-through capability lets the power module function as an uninterruptible power supply (UPS). In practice, however, the IC's internal architecture introduces a critical limitation: connecting or disconnecting the USB-C power source causes it to mechanically switch its internal power routing, producing an output voltage interruption of roughly 30 to 50 milliseconds [1][33].

This brief gap is negligible for something like a smartphone, which has its own internal battery to fall back on but it's fundamentally incompatible with a tethered single-board computer. The Raspberry Pi Zero 2 W simply lacks sufficient onboard bulk capacitance on its 5V rail to absorb such a gap. As a result, a 50 ms interruption is enough to drop the rail below the undervoltage lockout threshold, triggering a brownout and forcing a hard reboot with the risk of corrupting the active thermal imaging script or the SD card's file system.

As a result, true seamless UPS operation cannot be achieved with the bare IP5306 module alone. To preserve system stability and data integrity without modifying the power delivery hardware, a strict operational constraint was adopted instead: the system operates in two mutually exclusive states: active thermal monitoring, running strictly on battery power, and standby charging, plugged into the wall. The user must not connect or disconnect the USB-C charging cable while the operating system is booted, and the sensor is actively polling. If charging is needed, the correct procedure is to first initiate the hardware safe-shutdown sequence, wait for the microcontroller to fully power down, and only then connect the external power supply.

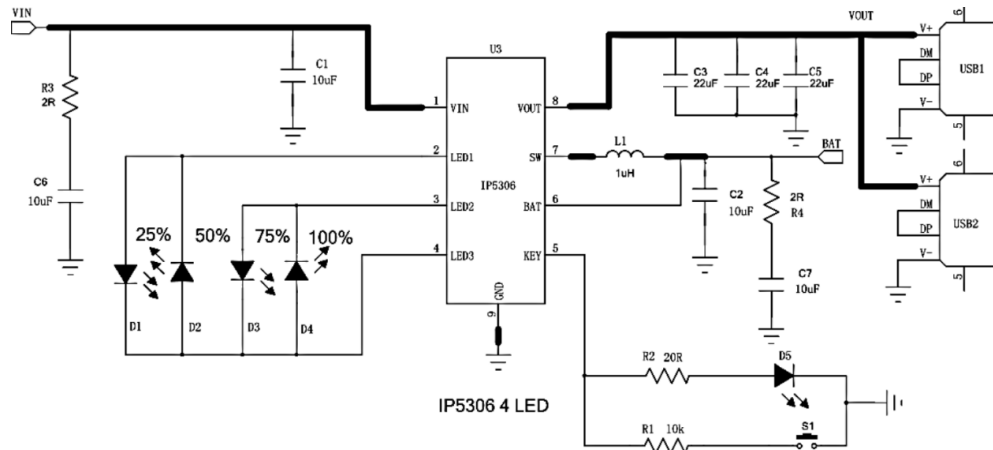


Figure 20. Typical application circuit and peripheral schematic for the IP5306 power management IC [33]

5.5 Visual Interface: 2.0-Inch IPS Display

The primary human-machine interface (HMI) for the thermal camera is a 2.0-inch IPS TFT LCD module. Unlike standard Twisted Nematic (TN) panels, which suffer from color inversion and contrast loss at off-axis viewing angles, IPS technology aligns the liquid crystal molecules parallel to the glass substrates. This architectural difference is what gives IPS panels their characteristic wide, 178-degree viewing angles and accurate color reproduction [34].



Figure 21. Front visual profile 8-pin interface configuration of the 2.0-inch IPS TFT module [34]

The display matrix itself consists of 240×320 active pixels a standard portrait QVGA resolution which offers an ample visual canvas for a handheld thermal diagnostic tool. However, since the MLX90640 natively captures a landscape 32×24 thermal image while the display's native orientation is portrait, the software stack performs a 90-degree coordinate rotation during the interpolation phase. This effectively transposes the display into a 320×240 landscape canvas, onto which the thermal feed is mapped using an exact 10x scale factor, filling the screen fully and producing a smooth, easily readable thermal gradient without harsh pixelation [34].

5.5.1 Controller Architecture and Color Rendering

Driving this array of 76,800 individual pixels requires a dedicated controller, and here the module relies on an embedded ST7789 single-chip display driver. This chip contains its own internal Graphic RAM (GRAM), large enough to hold a full frame of visual data, which offloads the burden of continuous screen refreshing from the host processor [34].

To keep the data transmission payload from the Raspberry Pi manageable, the display runs in a 16-bit RGB565 color format. Packing the red, green, and blue channel data into a single 16-bit integer per pixel cuts the transmission payload by 33% compared to true 24-bit color, or halves the memory bandwidth required relative to standard 32-bit padded structures. This efficient packing helps maximize the video refresh rate without any noticeable loss in the visual fidelity of the thermal colormaps [34].

5.5.2 System Implementation

To sustain the throughput real-time video rendering demands, the display is interfaced with the Raspberry Pi Zero 2 W via a unidirectional 4-wire SPI bus. Since SPI is a synchronous protocol without the addressing overhead of I2C, the Raspberry Pi can clock the bus aggressively, pushing the interpolated thermal matrix into the display's internal GRAM almost instantly [34].

The module connects through an 8-pin header. As shown in Figure 21, the hardware routing maps to the SBC as follows:

- GND: Tied to the common system ground.
- VCC: Connected to the primary 3.3V logic power rail.
- SCL: Synchronized to the processor's SPI clock output.
- SDA: Serves as the MOSI (Master-Out-Slave-In) line for image data.
- RES: Hardware reset, used to forcibly reinitialize the ST7789 controller on boot.
- DC: Data/Command toggle, switching the IC between configuration instructions and raw pixel data.
- CS: Active-low Chip Select, governing the display's frame reception.
- BLK: Backlight control, allowing the host to manage the LED array's brightness [34].

5.6 3D Enclosure

5.6.1 CAD Modeling

To protect the internal electronics and provide an ergonomic user experience, a custom 3D-printed enclosure was developed. The mechanical modeling was carried out in Autodesk Tinkercad, a cloud-based CAD environment chosen for its rapid prototyping capabilities.

The primary engineering constraint driving the chassis design was spatial efficiency. The internal volume had to be measured precisely enough to securely house the Raspberry Pi Zero 2 W, the power management module, and the bulky dual-18650 battery bank, while still leaving room for wire routing and passive thermal dissipation.

The assembled unit's overall external dimensions measure [141] mm × [75] mm × [54] mm.

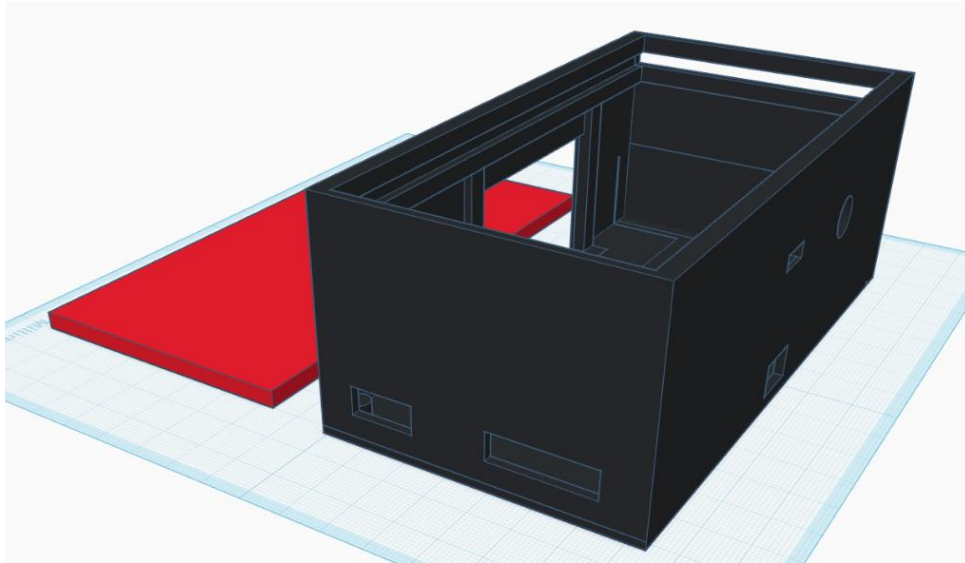


Figure 22. View of the 3D enclosure model in Autodesk Tinkercad

5.6.2 Interface Cutouts

The final enclosure takes the form of a rigid structural shell, optimized for 3D printing. The casing is built to strict physical tolerances, with dedicated cutouts positioned across its faces to separate the user interface, the environmental sensors, and the hardware safety controls into distinct zones:

- User Interface (Figure 23): Serving as the operator's main dashboard, this side features a custom bezel framing the 2.0-inch IPS display, along with two control mechanisms: a two-position toggle switch mapped to a GPIO pin for triggering the Pi's software-level safe-shutdown sequence, the SD card slot, and a master hardware switch that fully powers down the BMS.

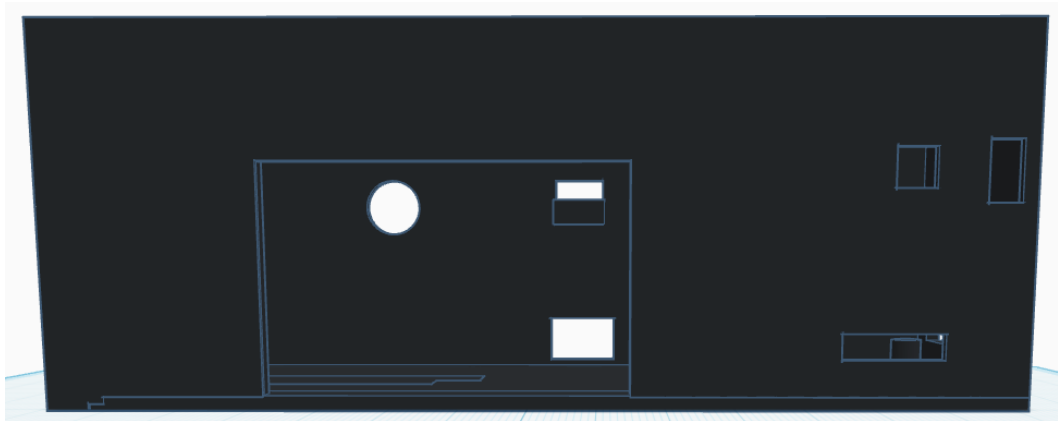


Figure 23. Primary Long Face

- Peripheral Access (Figure 24): One lateral side has structural openings aligned with the Raspberry Pi's native peripheral ports, allowing for direct debugging or the attachment of external devices without disassembling the shell.

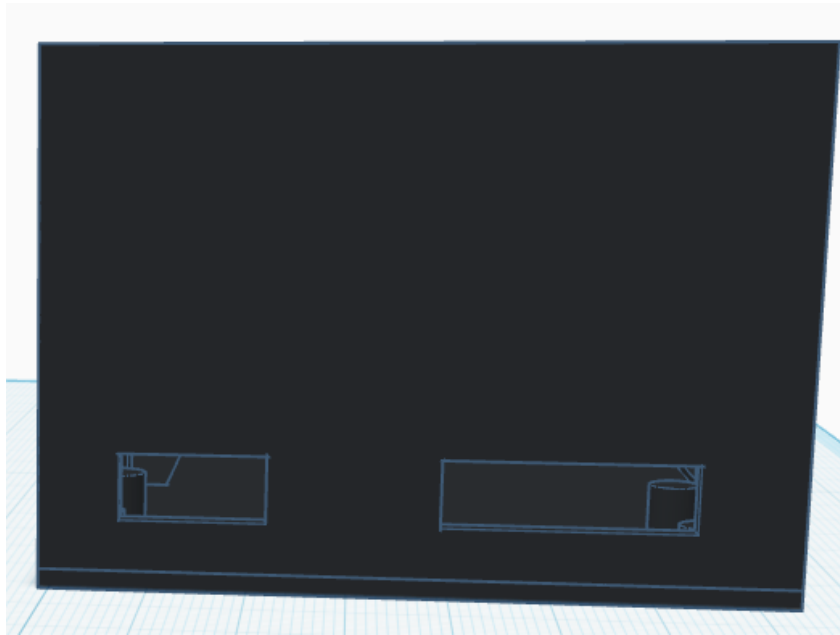


Figure 24. Short Face

- Environmental & Safety Interface (Figure 25): Facing the target environment, this side holds a precise circular aperture for the MLX90640 sensor and a cutout for the IP5306 module's USB-C charging input. It also includes an emergency hardware kill switch, which physically severs the power connection between the BMS and the Raspberry Pi, letting the user instantly isolate the logic board if a critical system lockup or thermal runaway occurs.

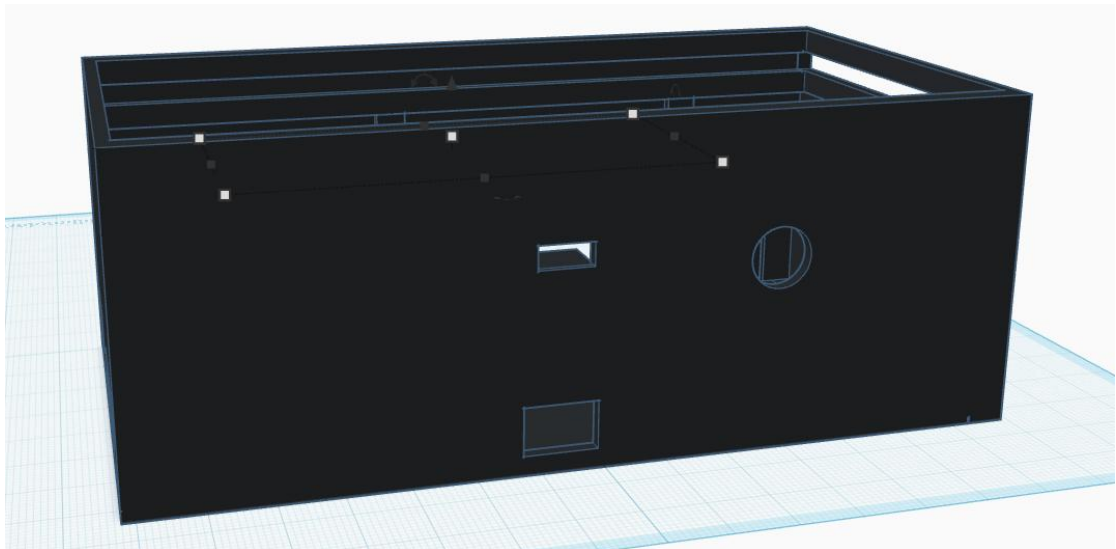


Figure 25. Opposing Long Face

5.7 Software Implementation

The software architecture of the thermal imaging system was developed entirely in Python. This language was selected as the primary implementation choice for its versatility, capacity for

rapid prototyping, and seamless integration with embedded Linux environments such as the Raspberry Pi OS. A particular advantage of Python in hardware-level engineering lies in its extensive, natively supported ecosystem of open-source libraries.

In this implementation, Python serves as the bridge between low-level hardware control and high-level data processing. It enables straightforward communication with hardware protocols (I²C and SPI) and General-Purpose Input/Output (GPIO) pins through the Adafruit-Blinka and Board abstraction layers, while simultaneously supporting the computationally intensive matrix operations required to process the sensor's 768 individual data points at high frame rates without bottlenecking the CPU, handled via the NumPy library. Libraries such as Pillow (PIL) and Matplotlib, in turn, manage the task of translating raw numerical temperature data into rendered, color-mapped visual graphics. Taken together, these tools allow the system to maintain a readable, modular codebase while managing hardware state and data pipelines concurrently.

The software execution flow was designed to run entirely autonomously, launching automatically upon system boot. To document its operation, the codebase is organized into distinct functional blocks, beginning with hardware state management.

5.7.1 Hardware Initialization and Bus Configuration

Before the system can acquire or process any thermal telemetry, the physical communication buses must first be instantiated and bound to their respective hardware peripherals. The software begins by configuring the Serial Peripheral Interface (SPI) bus to drive the ST7789 TFT display. Using the digitalio abstraction layer, specific Raspberry Pi GPIO pins are explicitly designated for the Chip Select (CE0), Data/Command (D24), and hardware Reset (D25) lines, and the display object is then initialized with a dedicated SPI baudrate of 24 MHz. This high clock frequency is a deliberate requirement, as it ensures the data bus has sufficient bandwidth to continuously push full 240×320 RGB565 image matrices to the screen without introducing visual lag or frame tearing.

Following the display configuration, the I²C bus is established on the standard Serial Clock (SCL) and Serial Data (SDA) pins to interface with the Melexis MLX90640 thermal sensor. As noted in the theoretical framework, the MLX90640 requires the host processor to extract a substantial payload of floating-point calibration constants from the sensor's internal EEPROM at boot. Embedded Linux environments, however, are subject to a known limitation regarding I²C clock stretching, which can occasionally cause this initial memory read to drop data packets and fail.

To address this hardware-level constraint, a fault-tolerant software initialization loop was implemented. Rather than attempting a single, linear connection that could crash the script on failure, the software runs a continuous try-except polling loop: the boot sequence is deliberately delayed by two seconds to allow the sensor's silicon to stabilize, after which the I²C bus is queried repeatedly. If the EEPROM extraction fails due to a clock stretching timeout, the script catches the exception, waits 10 milliseconds, and retries. This polling approach ensures that the software proceeds only once a complete, uncorrupted memory read has been confirmed.

Once the sensor object is successfully established, its internal refresh rate is fixed at 8 Hz. This value was chosen to balance a fluid video feed for the operator against the computational overhead the CPU needs to perform the matrix interpolation between frames.

```

spi = board.SPI()
cs_pin = digitalio.DigitalInOut(board.CE0)
dc_pin = digitalio.DigitalInOut(board.D24)
reset_pin = digitalio.DigitalInOut(board.D25)

disp = st7789.ST7789(
    spi, width=240, height=320,
    x_offset=0, y_offset=0, baudrate=24000000,
    cs=cs_pin, dc=dc_pin, rst=reset_pin
)

i2c = busio.I2C(board.SCL, board.SDA)
mlx = None

time.sleep(2)

while mlx is None:
    try:
        mlx = adafruit_mlx90640.MLX90640(i2c)
    except Exception:
        time.sleep(0.01)

mlx.refresh_rate = adafruit_mlx90640.RefreshRate.REFRESH_8_HZ

```

Figure 26. Hardware initialization and fault-tolerant I²C polling loop

5.7.2 Thermal Data Acquisition and Matrix Normalization

Once the hardware buses are initialized, the software enters its primary execution loop, continuously querying the sensor for new thermal telemetry. Raw data extraction is performed via the `getFrame()` method, which pulls the fully calculated temperature values from the sensor's RAM and populates a pre-allocated 768-element Python list. Since a one-dimensional array is unsuitable for graphical processing, NumPy is used to cast this raw list into an optimized array and reshape it into a two-dimensional 24×32 spatial matrix.

With the data spatially organized, the system extracts key diagnostic telemetry before generating the visual output. Using NumPy's underlying C-bindings, the software locates the minimum (`np.min`) and maximum (`np.max`) temperature values within the current frame essentially instantaneously, avoiding the latency that native Python for-loops would introduce. The algorithm also targets the center pixel at matrix index `[11, 15]`, which serves as a real-time focal measurement point for the operator.

To render this numerical data on a color display, the absolute temperature matrix must first be normalized into a relative scalar field between 0.0 and 1.0, which allows the dynamic range of the thermal map to adjust automatically to the hottest and coldest objects present in the frame. This is achieved through standard Min-Max feature scaling, a common image processing technique used to maximize visual contrast [35]:

$$N_{i,j} = \frac{T_{i,j} - T_{min}}{T_{max} - T_{min} + 0.01} \quad (4)$$

Here, $N_{i,j}$ is the normalized scalar value of a given pixel, $T_{i,j}$ its absolute temperature in Celsius, and T_{min} and T_{max} the global extremes of the current frame. The added constant of 0.01 in the denominator serves as a software fail-safe, preventing a divide-by-zero error in the event the sensor observes a perfectly uniform thermal field where T_{max} equals T_{min} .

This normalized matrix is then passed directly into the Matplotlib 'inferno' colormap, which converts each scalar value into an RGBA (Red, Green, Blue, Alpha) floating-point tuple. Because the ST7789 display cannot process floating-point colors or transparency values, the software strips the alpha channel, scales the RGB values by 255, and casts the resulting matrix into an 8-bit unsigned integer array (np.uint8), preparing the thermal data for final image interpolation and GUI rendering.

```
try:
    mlx.getFrame(frame)

    data = np.array(frame).reshape((24, 32))

    d_min = np.min(data)
    d_max = np.max(data)
    temp_centru = data[11, 15]

    norm = (data - d_min) / (d_max - d_min + 0.01)
    rgba = color_map(norm)
    rgb = (rgba[:, :, :3] * 255).astype(np.uint8)
```

Figure 27. Thermal data matrix manipulation and colormap normalization

5.7.3 Spatial Interpolation and Graphical User Interface (GUI)

With the absolute temperature data normalized and color-mapped, the resulting integer array is converted into a Pillow (PIL) image object for further structural modification. Because the physical orientation of the MLX90640 sensor on the printed circuit board does not intrinsically match the vertical alignment of the ST7789 display, the software must apply geometric transformations to the image matrix: the frame is rotated 90 degrees and horizontally inverted (FLIP_LEFT_RIGHT) so that the on-screen rendering accurately mirrors the operator's real-world perspective.

Once correctly oriented, the image exists at a native resolution of only 24×32 pixels. Rendered directly to the 240×320 screen, this would produce a grid of large, blocky squares rather than a coherent thermal image. To achieve the smooth gradients expected of professional thermography equipment, the software applies a bilinear spatial interpolation algorithm (Image.BILINEAR). When the image is scaled up by a factor of ten, this method avoids visible pixelation by calculating the weighted average of the four nearest neighboring pixels to generate smooth transition colors for each newly created pixel. As this interpolation requires millions of floating-point calculations per frame, it draws heavily on the quad-core architecture of the Raspberry Pi Zero 2 W to sustain a seamless video feed.

```

img = Image.fromarray(rgb, 'RGB')
img = img.rotate(90, expand=True)
img = img.transpose(Image.FLIP_LEFT_RIGHT)
img = img.resize((240, 320), Image.BILINEAR)

draw = ImageDraw.Draw(img)

txt_max = f"Max: {d_max:.1f} C"
txt_min = f"Min: {d_min:.1f} C"
draw.rectangle([(5, 5), (95, 35)], fill=(0, 0, 0))
draw.text((10, 8), txt_max, font=font, fill=(255, 100, 100))
draw.text((10, 20), txt_min, font=font, fill=(100, 150, 255))

cx, cy = 120, 160
draw.line([(cx - 5, cy), (cx + 5, cy)], fill=(255, 255, 255), width=1)
draw.line([(cx, cy - 5), (cx, cy + 5)], fill=(255, 255, 255), width=1)

txt_centru = f"{temp_centru:.1f} C"
draw.rectangle([(cx + 8, cy + 8), (cx + 55, cy + 22)], fill=(0, 0, 0))
draw.text((cx + 10, cy + 10), txt_centru, font=font, fill=(255, 255, 255))

disp.image(img)

```

Figure 28. Bilinear spatial interpolation and GUI overlay generation

Following the spatial upscaling, the software overlays a real-time diagnostic Heads-Up Display (HUD) using the ImageDraw module. To give the operator immediate thermal context, the previously calculated minimum and maximum frame temperatures are rendered in the upper-left quadrant of the display. To ensure alphanumeric legibility regardless of the shifting thermal colors behind it, the software draws an opaque black rectangular bounding box beneath the text layer to preserve contrast.

Finally, the GUI generates a targeting reticle at the mathematical center of the 240×320 display ($X=120$, $Y=160$), formed by two 10-pixel white intersecting lines that create a precision crosshair. The absolute Celsius value of the targeted object, extracted earlier from matrix index [11, 15], is rendered adjacent to the reticle within its own high-contrast bounding box. Once this final overlay is drawn into the software buffer, the completed, multi-layered frame is pushed over the SPI bus to the ST7789 display controller via the `disp.image(img)` command, completing the thermal processing loop.

5.7.4 Hardware Interrupts and Safe System Shutdown

Unlike standard microcontrollers that execute firmware entirely from flash memory, the Raspberry Pi Zero 2 W runs a full embedded Linux kernel with an ext4 filesystem mounted on a micro-SD card. In such architectures, abruptly cutting power while the operating system is actively reading, writing, or swapping memory will typically result in partition corruption and boot failure. To protect against this and ensure the long-term reliability of the diagnostic instrument, a dedicated hardware-to-software shutdown mechanism was implemented.

The physical interface consists of a tactile hardware switch connected to GPIO pin 21 (Physical Pin 40). At software initialization, this pin is configured as a digital input with an internal pull-up resistor (Pull.UP), which ensures the pin reads a logical HIGH by default and

switches to a logical LOW when the switch bridges the circuit to ground. To make the system position-agnostic, allowing the device to be powered on regardless of the switch's initial physical position, the script records the switch's state (`stare_initiala_buton`) into memory at the moment the camera boots.

```
buton = digitalio.DigitalInOut(board.D21)
buton.direction = digitalio.Direction.INPUT
buton.pull = digitalio.Pull.UP

stare_initiala_buton = buton.value
```

Figure 29. GPIO hardware switch initialization and state recording

During the primary execution loop, the software polls GPIO 21 at the start of every frame. If the current pin value deviates from the recorded boot state, the software interrupts the thermal processing pipeline and initiates a safe shutdown sequence.

To provide immediate visual feedback to the operator, the software generates a blank black image matrix, renders a red warning prompt ("POWER OFF"), and pushes this final static frame to the SPI display. With the graphical interface locked, the Python script issues a low-level operating system call via the `os.system` library, passing the command `sudo shutdown -h now` directly to the Linux kernel. This command halts all CPU processes, unmounts the filesystem gracefully, and signals the power management IC to cut voltage to the processor safely. The script then enters a 20-second thread suspension (`time.sleep(20)`), which deliberately freezes the Python loop to ensure it does not attempt to query the I²C bus or write to the SPI display while the kernel is still in the process of powering down the hardware.

```
while True:
    if buton.value != stare_initiala_buton:
        img_stop = Image.new("RGB", (240, 320), (0, 0, 0))
        draw_stop = ImageDraw.Draw(img_stop)
        draw_stop.text((70, 150), "POWER OFF", font=font, fill=(255, 50, 50))
        disp.image(img_stop)

        os.system("sudo shutdown -h now")

        time.sleep(20)
```

Figure 30. Hardware interrupt polling and OS safe shutdown execution

6 Experimental results

Following the theoretical design, hardware assembly, and software implementation phases, the project culminated in a fully functional, portable thermal imaging prototype. Before subjecting the device to quantitative diagnostic testing, a complete physical overview of the assembled instrument is necessary to confirm the integration of the internal electronics, the custom $141 \times 75 \times 54$ mm 3D-printed enclosure, and the user interface.

The final assembly was designed with a focus on ergonomic simplicity and standalone operation, eliminating the need for external peripherals or SSH connections during normal use. The operator interface is concentrated on the rear face of the device. As shown in Figure 31, this panel houses the 2.0-inch IPS TFT display, which provides a high-contrast visual output of the thermal environment.

Adjacent to the screen are two control interfaces. The first is the hardware interrupt toggle switch which allows the user to trigger the safe Linux shutdown sequence, protecting the SD card filesystem from corruption. The second is a tactile button linked to the internal IP5306 Battery Management System (BMS); this allows the operator to check the 4000mAh battery's charge state via indicator LEDs, and can also be used to manually wake or sleep the BMS power output.

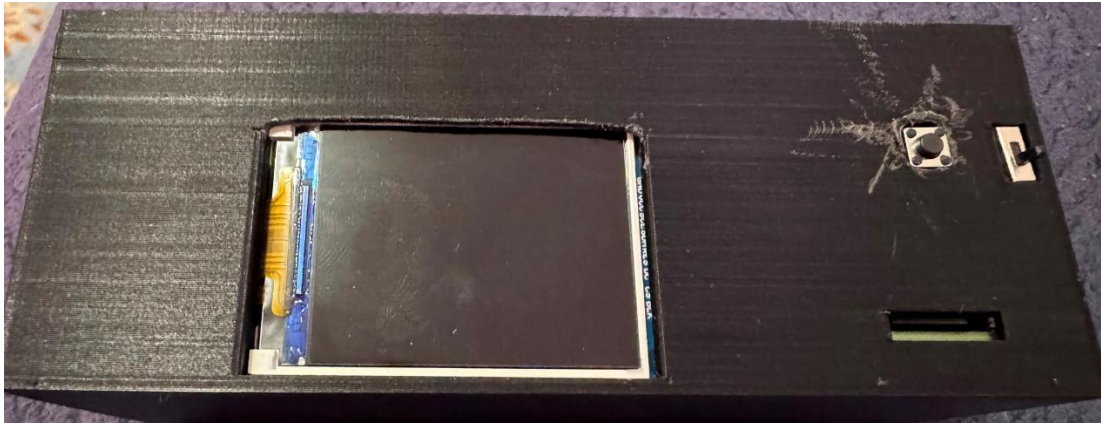


Figure 31. Rear view of the assembled thermal camera

The environment-facing side of the instrument is designed to maximize sensor exposure while facilitating charging. As illustrated in Figure 32, the Melexis MLX90640-BAB thermal sensor is mounted flush with the exterior chassis, an unobstructed placement that ensures the sensor's narrower, focused $55^\circ \times 35^\circ$ Field of View (FOV) is not clipped by the edges of the 3D-printed casing.

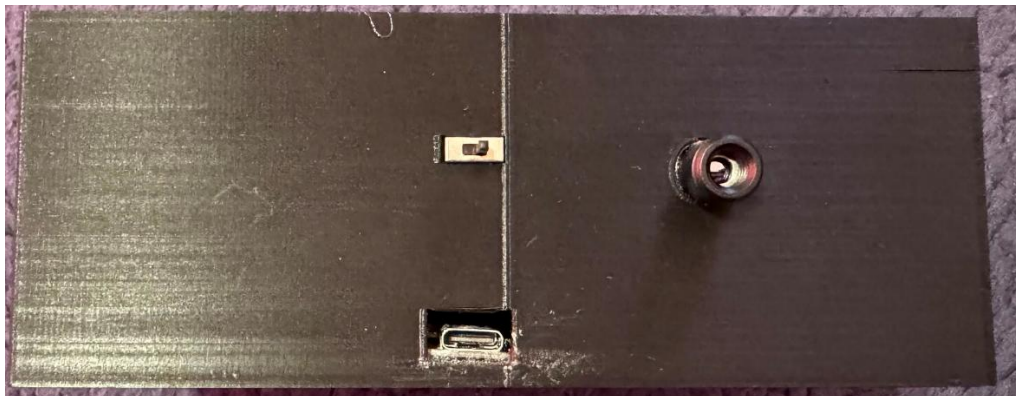


Figure 32. Front view of the assembled thermal camera

This front panel also houses the primary hardware controls for the electrical circuit. A principal on/off toggle switch is integrated here to physically connect or sever the main battery power, functioning as a master hardware disconnect. Bellow it, the IP5306's USB-C port is exposed, allowing the dual 18650 parallel Lithium-ion cells to be recharged without disassembling the enclosure.

With the physical assembly finalized and all hardware controls verified to be functioning correctly, the system was ready for quantitative evaluation. The following sections detail the experimental testing of the camera, focusing on its computational boot performance, battery autonomy, environmental adaptability, and the diagnostic accuracy of its thermal telemetry.

6.1 System Performance and Power Autonomy

To validate the standalone capabilities of the thermal camera, the system's computational responsiveness and electrical endurance were quantitatively evaluated. Because the device is intended for quick-deployment diagnostic scenarios, boot time, processor stability, and battery life are critical performance metrics.

6.1.1 Computational Performance and Boot Sequence

Unlike standard microcontrollers, the Raspberry Pi Zero 2 W must load a complete Linux kernel before executing the Python thermal imaging script. When the principal hardware switch is toggled, the IP5306 power management IC immediately supplies the 5.15V baseline power to the single-board computer. The system requires 32 seconds to complete the headless boot sequence, initialize the SPI and I²C buses, extract the EEPROM calibration constants from the MLX90640, and push the first interpolated visual frame to the ST7789 display, a deployment time that is competitive for a Linux-based embedded diagnostic tool.

Once booted, system performance remains stable. To monitor resource allocation without interfering with the graphical interface, a remote SSH diagnostic session was established. As documented in Figure 33, the quad-core ARM Cortex-A53 processor handles the continuous thermal processing pipeline with considerable computational overhead to spare.



Figure 33. Remote SSH detailing active CPU load, RAM allocation, and system uptime during continuous thermal monitoring

While calculating the bilinear interpolation needed to up-scale the 768-pixel array to 76,800 pixels at 8 frames per second, total CPU utilization stabilizes at merely 12%. The Python script and operating system together consume only 0.13 GB (approximately 31%) of the available 512 MB LPDDR2 RAM. The internal SoC temperature was recorded at a stable 46°C after 40 minutes of continuous uptime inside the 3D-printed enclosure, well below the 80°C thermal throttling threshold, and evidence that the Raspberry Pi Zero 2 W provides the floating-point bandwidth required without generating excess heat capable of skewing the adjacent sensor's readings.

6.1.2 Power Consumption and Battery Autonomy

A primary goal of the implementation phase was to ensure the device could sustain extended diagnostic sessions without requiring grid power. To calculate the autonomy of the system, the

energy capacity of the parallel 18650 battery bank was mapped against the active electrical loads of the microprocessor and display.

The complete electrical parameters, including current draw and Watt-hour (Wh) energy values, are summarized in Table 5.

Table 5. System Power Budget and Energy Storage Parameters

System Component / Parameter	Operating Voltage	Current Draw (mA)	Power (W) / Energy (Wh)
Active Subsystems			
Raspberry Pi Zero 2 W (Active Load)	5.0V	350 mA	1.75 W
ST7789 IPS Display	3.3V	40 mA	0.20 W
MLX90640-BAB Sensor	3.3V	20 mA	0.10 W
Total System Load	5.0V	410 mA	2.05 W
Energy Storage			
Dual 18650 Battery Bank	3.7V	4000 mAh	14.80 Wh
IP5306 Boost Converter Efficiency	5.0V	-	91%

Using the data in Table 5, the total autonomous runtime (T_{run}) of the thermal camera can be projected using an energy-conversion formula. Multiplying the battery's raw capacity (C_{bat}) by its nominal voltage (V_{bat}) and the boost converter's efficiency (η), then dividing by the total system power requirement ($V_{sys} \cdot I_{sys}$), yields:

$$T_{run} = \frac{C_{bat} \cdot V_{bat} \cdot \eta}{V_{sys} \cdot I_{sys}} = \frac{4000 \cdot 3.7 \cdot 0.91}{5.0 \cdot 410} = 6.5 \quad (5)$$

It is important to note that this 6.5-hour projection represents a theoretical minimum runtime, calculated using the absolute peak current draw of all components. In real-world empirical testing, the fully charged device demonstrated an extended operational autonomy ranging between 7 and 8 hours.

This real-world increase in battery endurance is directly attributed to the dynamic power scaling of the Raspberry Pi Zero 2 W's ARM architecture. While the initial Linux boot sequence and sensor initialization demand peak current from the boost converter, the steady-state thermal processing pipeline is highly optimized. Because the NumPy matrix mathematics consume only 12% of the CPU's total capacity, the processor dynamically lowers its voltage and frequency demands during normal operation. Consequently, the real-world steady-state current draw falls significantly below the estimated 410 mA peak, allowing the system to maximize the efficiency of the 14.80 Wh battery bank. This extended 7-to-8-hour runtime heavily exceeds the standard operational requirements for handheld diagnostic tools, thoroughly validating the custom power delivery architecture. Furthermore, the 2.1A charging capability of the IP5306 allows the battery bank to be completely replenished via the USB-C port in less than 2.5 hours when the device is placed in standby mode.

6.2 Testing and Use Cases

To validate the real-world utility of the MLX90640-BAB sensor and the Python-based image processing pipeline, the thermal camera was tested across multiple physical environments. These tests were designed to quantitatively evaluate the system's spatial resolution, its biometric

sensitivity, and its ability to dynamically adjust the Matplotlib 'inferno' colormap using the programmed Min-Max normalization algorithm.

6.2.1 Case Study 1: Electronic Component Inspection

The primary engineering application for a portable thermal imaging system is the non-destructive inspection of electrical circuits and localized heat dissipation. In this initial test, the camera was directed at a laptop computer following a period of active use.

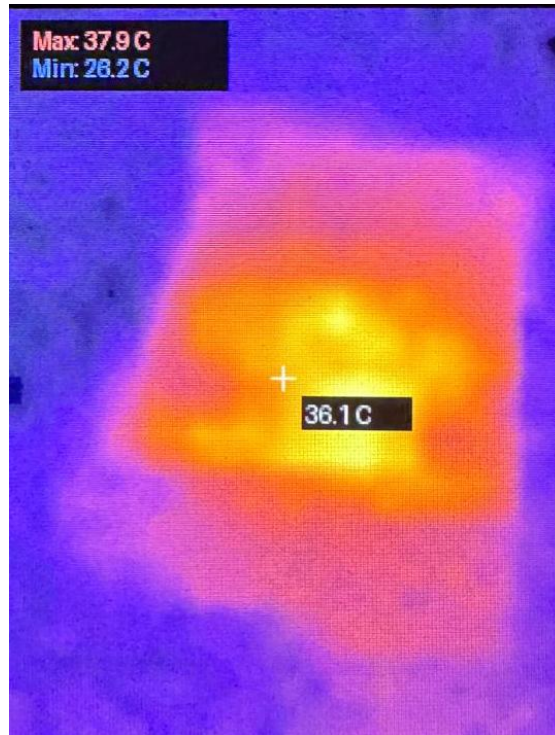


Figure 34. Thermal footprint of an active laptop computer

As demonstrated in Figure 34, the system successfully isolates the localized heat signatures radiating from the internal hardware. The Display establishes a baseline ambient room temperature (Min: 26.2°C) while identifying the absolute hottest point on the chassis at 37.9°C. The central crosshair, aimed directly at the upper keyboard area, dynamically measures a localized reading of 36.1°C. Because the MLX90640-BAB variant features a narrower $55^{\circ} \times 35^{\circ}$ Field of View (FOV), the camera provides excellent spatial density, allowing the operator to clearly distinguish the distinct thermal zones of the screen and the upper keyboard without wide-angle distortion.

6.2.2 Case Study 2: Temperature Extremes (Hot and Sub-Zero)

To evaluate the upper and lower limits of the array's absolute temperature measurement capabilities, the camera was tested against isolated hot and cold thermal masses.

In Figure 35, the camera successfully registers the extreme thermal output of the water. The system isolates a maximum frame temperature of 53.7°C, with the central crosshair confirming the targeted liquid at 52.0°C against an ambient 29.2°C background.

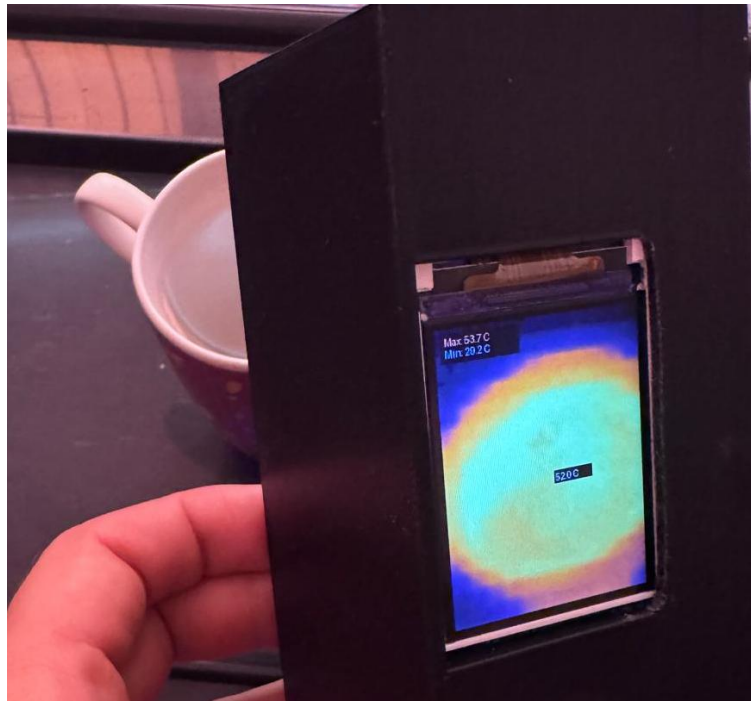


Figure 35. High-temperature liquid mass

Immediately following this, the system was directed at a bag of frozen ice to verify the Python script's ability to handle negative floating-point numbers.

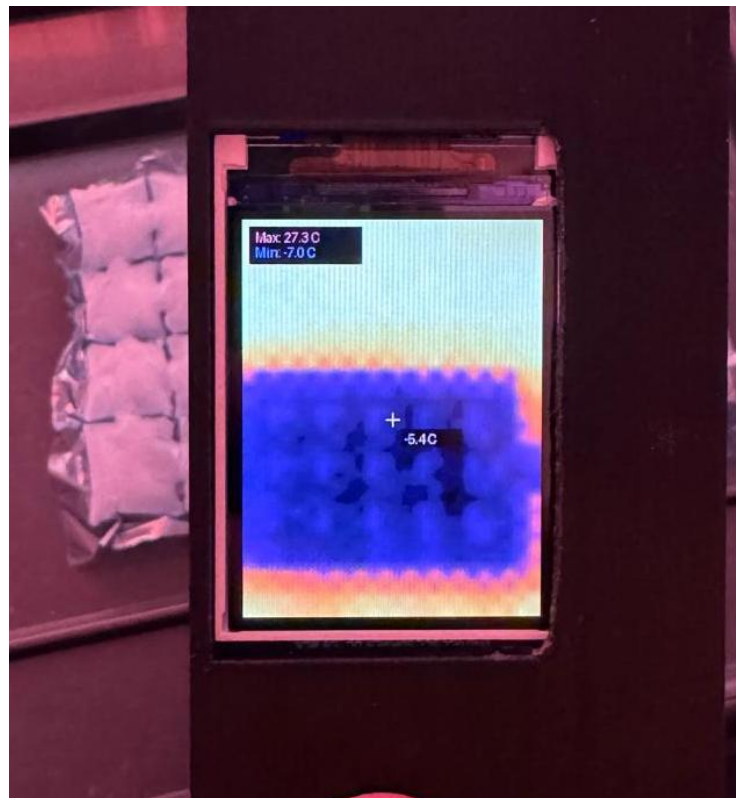


Figure 36. Sub-zero thermal measurement

As shown in Figure 36, the sensor easily captures the sub-zero target. The display perfectly updates to show an ambient room temperature of 27.3°C (Max) while the absolute coldest point drops to -7.0°C (Min), with the central crosshair at the ice at -5.4°C. Most importantly, this

proves that the NumPy np.min bindings and the GUI rendering logic do not crash when processing negative integer values.

6.2.3 Case Study 3: High-Contrast Dynamic Normalization

A critical software requirement was the system's ability to mathematically adapt to shifting thermal extremes simultaneously. To test the robustness of the normalization algorithm, both the cup of hot water and the bag of ice were placed within the camera's FOV.

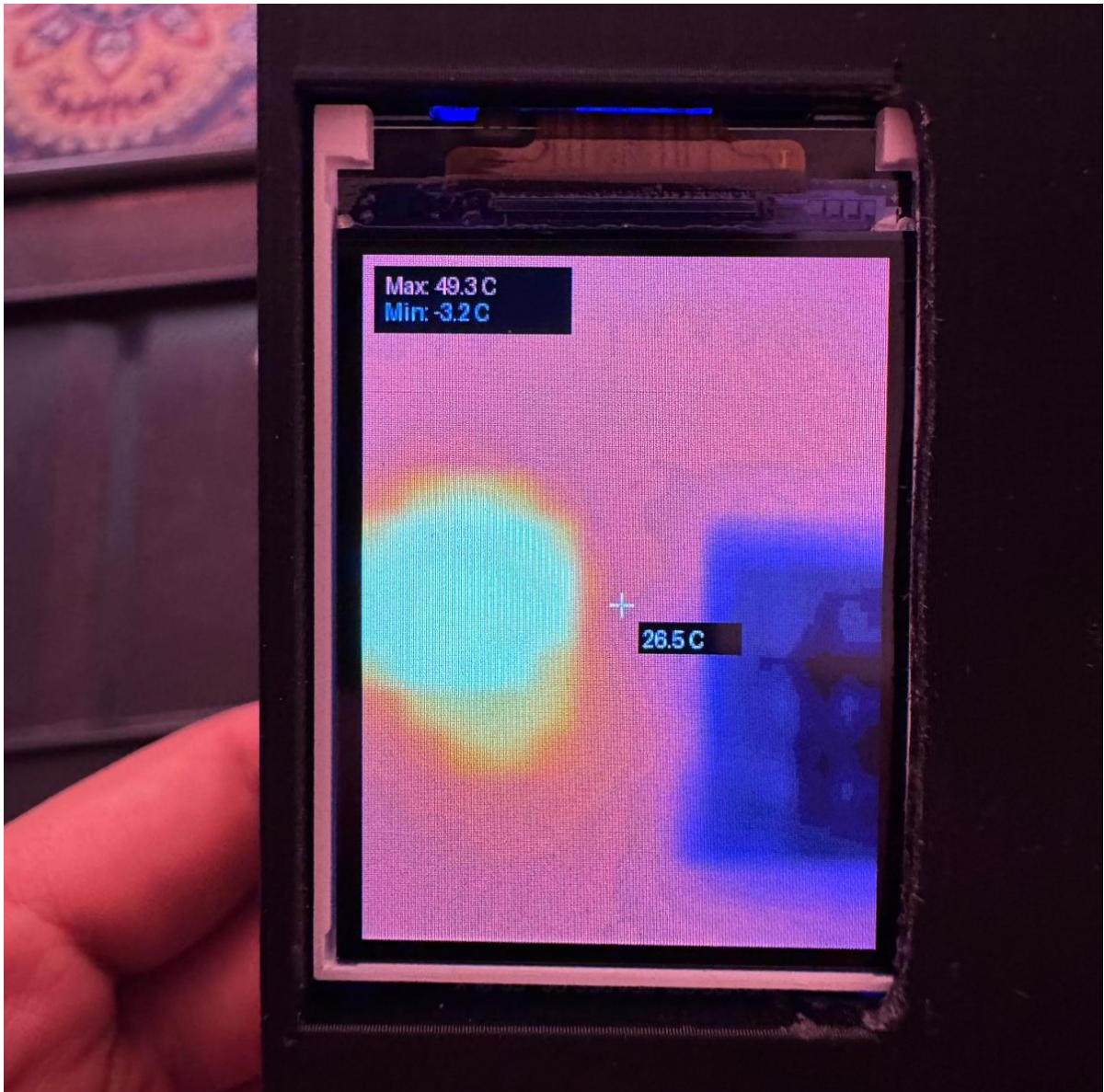


Figure 37. High-contrast environment testing

Figure 37 perfectly validates the custom Min-Max scaling formula implemented in the software architecture. Facing a massive thermal delta, the HUD extracts a maximum of 49.3°C and a minimum of -3.2°C within the exact same frame. The algorithm dynamically stretches the 'inferno' colormap across this 52.5°C gap: the hot water is saturated at the absolute top of the visual spectrum (yellow-orange), the ice is pushed to the absolute bottom (blue-violet), and the ambient background is rendered smoothly in the middle (red-pink). This test proves that the

mathematical fail-safes strictly prevent visual artifacting, even under extreme high-contrast conditions.

6.2.4 Case Study 4: Biometrics and Zero-Light Operation

Because infrared thermopiles detect emitted thermal radiation (heat) rather than reflecting visible light photons, true thermal cameras operate entirely independently of ambient lighting. To demonstrate this capability alongside the sensor's high-resolution spatial upscaling, the camera was used to capture a human hand in complete darkness.



Figure 38. Biometric thermal capture in a zero-light environment

As seen in Figure 38, the absence of visible light has zero impact on the telemetry. The camera captures the biometric heat signature with incredible clarity, registering a maximum skin temperature of 34.1°C against a 26.1°C ambient background. The effectiveness of the Raspberry Pi's bilinear interpolation is highly evident here: the individual fingers are perfectly distinguishable, with smooth, continuous thermal gradients transitioning across the skin surface. This clearly confirms that both the physical thermal sensor and the software scaling algorithms are functioning exactly as designed, successfully detecting and rendering even subtle variations in body heat.

6.3 Comparative Analysis

To establish the scientific validity and measurement fidelity of the custom thermal imaging prototype, its performance was benchmarked against a calibrated, professional-grade reference instrument. For this evaluation, the FLIR ETS320 (a high-precision benchtop thermal imaging

system widely utilized in industrial electronics diagnostics) was chosen as the baseline measurement standard.

6.3.1 Experimental Methodology

Comparative testing was conducted using a soldering iron as a dynamic thermal source. Both the custom prototype and the FLIR ETS320 were positioned at an identical distance, simultaneously targeting the heating element as it ramped from ambient room temperature to high temperature.

When benchmarking these systems, a fundamental distinction in spatial resolution must be addressed. The FLIR ETS320 utilizes a dense 320×240 microbolometer array, allowing its microscopic pixels to independently isolate the razor-sharp tip of the iron. Conversely, the prototype's smaller 32×24 MLX90640 array covers a much larger spatial angle per pixel. A single pixel targeting the iron's tip unavoidably integrates both the concentrated heat of the metal and the cooler surrounding air, resulting in localized spatial averaging.

Because this physical limitation naturally lowers pinpoint peak readings, evaluating the prototype solely on microscopic focal targets misrepresents its overall accuracy. Therefore, this analysis benchmarks the dynamic frame scale limits. This scientifically rigorous approach directly verifies that the prototype accurately captures the global thermal gradient of the scene in parity with the commercial standard.

6.3.2 Visual and Quantitative Evaluation Across Thermal Stages

The dynamic heating sequence was evaluated across three distinct thermal stages: initial heating, intermediate stabilization, and high-temperature operation.

Stage 1: Initial Heating Phase

During the initial warming phase, the soldering iron began dissipating heat into the ambient workspace.

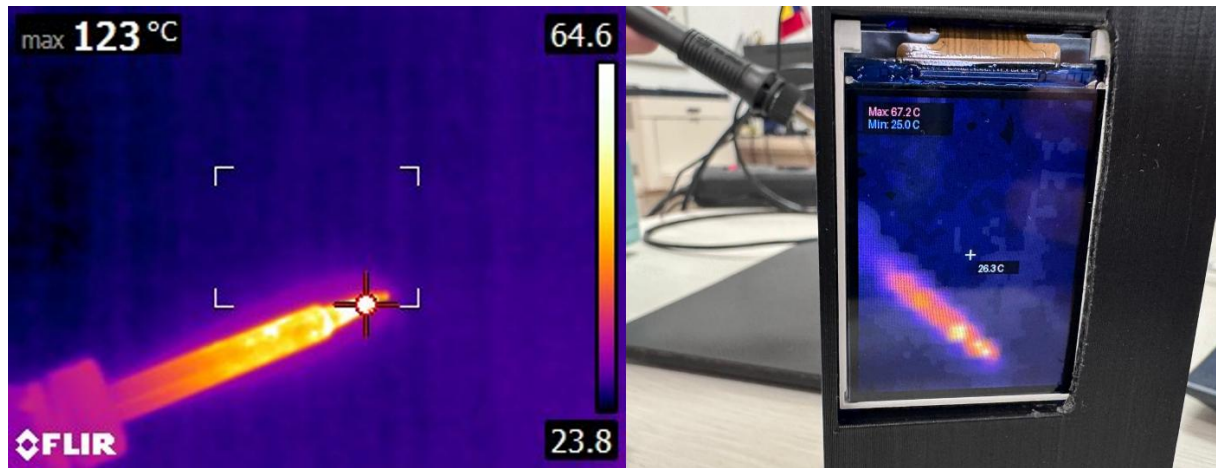


Figure 39. Side-by-side thermal comparison during the initial heating phase

As demonstrated in Figure 39, the prototype tracks the initial thermal rise with remarkable precision. The FLIR ETS320 established an ambient floor of 23.8°C and a visual scale maximum of 64.6°C . Simultaneously, the custom prototype reported an ambient baseline of 25.0°C and a frame maximum of 67.2°C . The correlation between the two systems is exceptionally close, exhibiting a variance of only 1.2°C at the lower boundary and 2.6°C at the upper boundary.

Stage 2: Intermediate Heating Phase

As the heating element reached typical low-melting-point solder ranges, the thermal gradient expanded significantly.

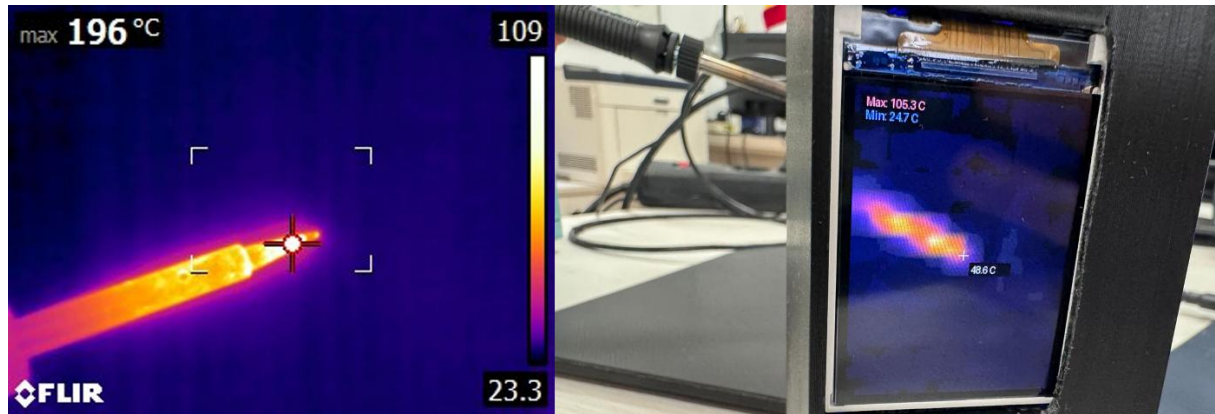


Figure 40. Side-by-side thermal comparison during the intermediate heating phase

Figure 40 illustrates the intermediate test stage. The FLIR system registered a dynamic range spanning from 23.3°C to 109.0°C. The custom device recorded a minimum ambient floor of 24.7°C and a peak frame reading of 105.3°C. Despite the substantial increase in radiative energy, the prototype's maximum scale diverged by only 3.7°C from the calibrated FLIR standard, proving strong linearity in the MLX90640's ADC conversion.

Stage 3: High-Temperature Operational Phase

In the final phase, the soldering iron was brought to standard operational working temperatures.

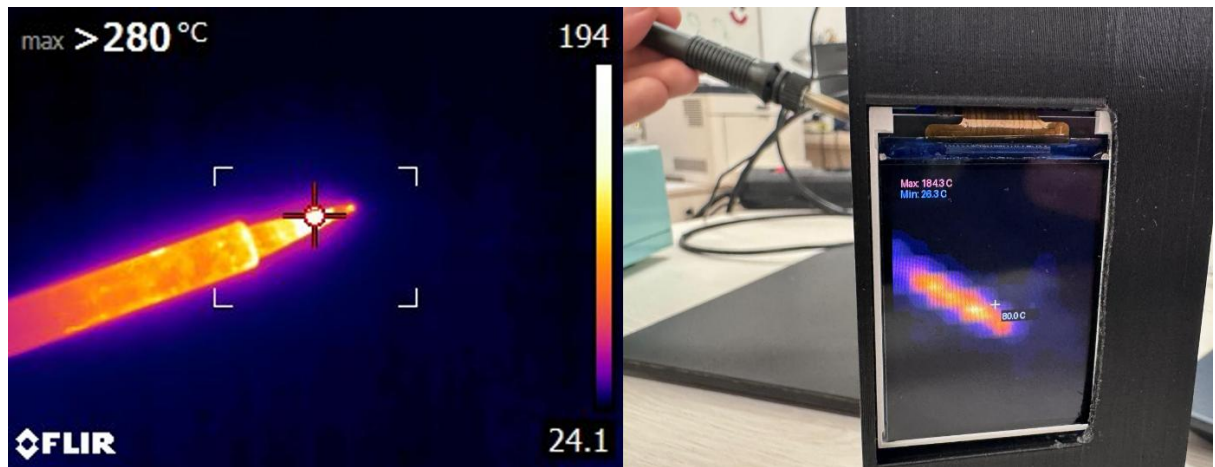


Figure 41. Side-by-side thermal comparison at high operational temperature

At high operational temperatures, shown in Figure 41, the FLIR scale extended from 24.1°C to 194.0°C. The prototype dynamically adjusted its upper limit to 184.3°C while holding the ambient baseline at 26.3°C. The slight divergence at this elevated tier ($\approx 9.7^\circ\text{C}$) is consistent with expected radiant heat loss and spatial averaging across the wider thermopile footprint at extreme gradients, yet the prototype maintains robust proportionality without saturating the sensor or distorting the 'inferno' visual colormap.

6.3.3 Dynamic Thermal Tracking and Linearity

To see how well the prototype performs as temperatures rise, maximum frame temperatures from both cameras were recorded at six different heating stages. This data was plotted into a chart to see if the custom camera's readings stayed aligned with the commercial FLIR camera.

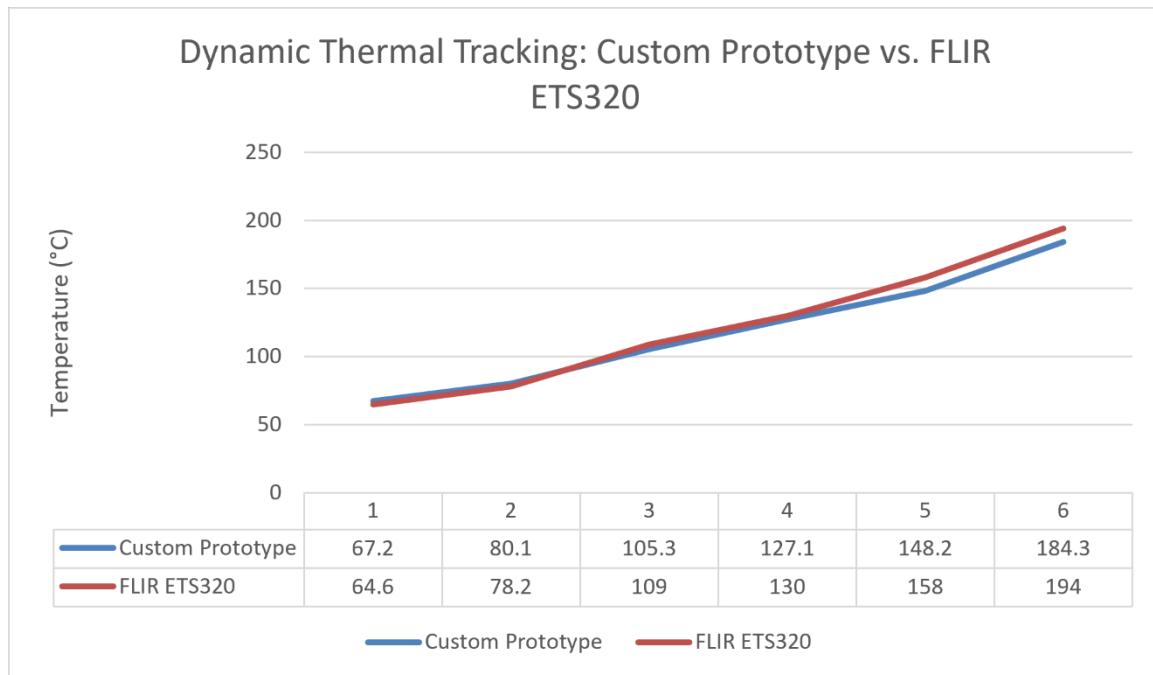


Figure 42. Temperature Comparison Chart

As shown in Figure 42, the custom camera tracks the temperature rise beautifully. During the first four measurements (from roughly 60°C up to 130°C), the prototype closely matched the FLIR system. The custom device recorded maximums of 67.2°C, 80.1°C, 105.3°C, and 127.1°C, which closely followed the FLIR's baseline readings of 64.6°C, 78.2°C, 109.0°C, and 130.0°C.

As the soldering iron heated up toward 200°C (measurements 5 and 6), a small, predictable gap appeared between the two lines. The FLIR reached 158.0°C and 194.0°C, while the prototype measured 148.2°C and 184.3°C. This difference (roughly 10°C at the peak) is not a software error. Because the MLX90640 sensor has physically larger pixels, it mixes the extreme heat of the tiny iron tip with the cooler air around it, resulting in a slightly lower reading.

Most importantly, the upward slope of both lines in the graph is almost identical from start to finish. This proves that despite being built with low-cost components, the custom Raspberry Pi prototype is a highly reliable and accurate tool for thermal diagnostics.

7 Conclusions

The main goal of this thesis was to design, build, and test a portable thermal camera for inspecting electronics. Turning this idea into a real device meant combining an MLX90640-BAB infrared sensor with the computing power of a Raspberry Pi Zero 2 W. The system works step by step: it reads heat data through a fast connection, processes the numbers using Python, and turns them into a live video. A major focus was keeping the software fast so the video would run smoothly, even when looking at very hot or very cold objects at the same time.

To make the software run better, special math and image-smoothing techniques were added to the code. By using the NumPy library, the system avoids slow Python calculations and quickly processes the 32x24 thermal data. Also, making the image bigger and smoother was very important for getting a clear picture. This smoothing removes blocky pixels, which helps a lot when looking at small temperature differences, like body heat. Along with a custom scaling formula, the software automatically adjusts the screen colors when the temperature changes suddenly, all without crashing or causing errors.

The device was made easy to hold and use by creating a custom 3D-printed case and adding a 2.0-inch screen. This step was a big achievement because it turned the project from a bundle of wires into a real, standalone tool. The user can see the thermal video on a digital display that shows the exact temperature using a center crosshair. Also, physical buttons were connected directly to the system. These buttons allow the user to safely turn off the device and manage the battery without needing a keyboard or a computer connection.

Testing the final camera proved that the system works very well, especially when it comes to battery life and temperature accuracy. By using an IP5306 charging chip and a large 4000mAh battery pack, the device can run non-stop for over seven hours. This means it can be used anywhere without being plugged into a wall. In addition, comparing the custom camera to an expensive FLIR ETS320 camera showed that the prototype is very accurate. The device successfully tracked rising temperatures and proved that good software can make up for the limits of a smaller, cheaper thermal sensor.

In the future, this project could be improved in several ways. One major upgrade would be using a better infrared sensor with more pixels. This would give clearer details and allow the camera to measure small hotspots from farther away. On the software side, adding a feature to take pictures with the press of a button would let users save images to the SD card for later review. Also, since the Raspberry Pi has built-in Wi-Fi, the camera could be updated to stream the thermal video over the internet. This would allow people to safely monitor dangerous electrical panels from a distance.

Finally, during the writing of this thesis, artificial intelligence tools were used only as helpful assistants. These tools helped with writing basic code, organizing research ideas, and fixing the flow of the text. However, all information, code, and ideas generated by AI were carefully checked, tested, and verified before being added to the final paper.